RATHINAM TECHNICAL CAMPUS (AUTONOMOUS)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

22EC603 – ADVANCED EMBEDDED SYSTEMS LABORATORY

Register Number : _____

Name of the Student : _____

Class : _____

Academic Year : _____

BONAFIDE CERTIFICATE

REGISTER NO. : ……..…….

STUDENT NAME : ……..…….

YEAR/SEMESTER : ……..…….

BRANCH : ……..…….

ACADEMIC YEAR : ……..…….

This is certified that this record is the bonafide work carried out by the above mentioned student in the …….. Laboratory during the Academic Year 2025-2026.

Staff In-Charge **Head of the Department**

Submitted for the Practical Examination held on ……..

Internal Examiner **External Examiner**

DO's AND DON'Ts NEED TO BE FOLLOWED IN THE LABORATORY

DO's

- Proper dress code has to be followed while entering the lab.
- Students should carry lab observation book and record in all aspects.
- Read and understand how to carry out an experiment thoroughly before coming to the lab and read the instructions and follow the instructions before working on the instruments.
- Enter your batch number and other details in the slip.
- Students should be in their allotted table and should not move around unnecessarily.
- Switch off the power supply while making any connection changes on the circuit.
- Report any broken plug, apparatus or expose damaged electrical wires to the faculty member/Lab technician immediately.
- Practical results should be noted down into their observation and result must be shown to the faculty member for verification
- After completing the experiments, students should return the component and apparatus and keep the chairs in proper place

DONT's

- Entering the laboratory without Identity Card, Observation, and record note book.
- Leaving the lab without any prior permission of the Faculty/Lab Instructor.
- Handling any equipment without reading the instruction manual.
- Changing their allotted seating or Computers with other students without getting permission from Faculty/Lab Instructor.
- Removing or exchanging the Keyboard / Mouse / Monitor connected to the assigned PC without permission.
- Using mobile phones while working in the laboratory.

VISION OF THE INSTITUTION

To be a leading and path-breaking Institution in multi-disciplinary education, research and industry related development for meeting the challenges of a New India.

MISION OF THE INSTITUTION

M1: Provide quality Engineering Education, Foster Research and Development; inculcate innovation in Engineering and Technology through state-of-the-art infrastructure.

M2: Nurture young men and women capable of assuming leadership roles in the society for the betterment of the country.

M3: Collaborate with industry, government organizations and society for curriculum alignment and focused, relevant outreach activities.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VISION OF THE DEPARTMENT

To achieve academic excellence in Electronics and Communication Engineering by imparting in depth knowledge to the students, facilitating research activities and cater to the ever changing industrial demands and societal needs.

MISSION OF THE DEPARTMENT

M1: To enhance the research knowledge of graduates along with updation of technological skills.

M2 : To provide technical expertise and to encourage innovative projects catering to the needs of the society.

M3: To impart quality education in Electronics and Communication Engineering through collaboration with industry.

PROGRAM EDUCATIONAL OUTCOMES (PEOs)

Graduates of the program B.E. Electronics and Communication Engineering will be able to:

PEO1	To prepare students to critically analyze existing literature in an area of specialization and enhance research to ethically develop innovative and technology-oriented methodologies with industrial support to solve the problems identified.
PEO2	To provide students with strong foundational concepts advanced techniques and tools in order to enable them to build solutions for societal needs.
PEO3	To enable graduates to pursue research and gain a successful career in academia through industrial association.

PROGRAM OUTCOMES (POs)

Graduates of the program B.E. Electronics and Communication Engineering will be able to:

PO1	**Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems
PO2	**Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	**Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	**Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	**Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	**The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	**Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	**Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	**Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	**Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	**Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES (PSOs)

Graduates of the program B.E. Electronics and Communication Engineering will be able to:

PSO1	Adapt information and communication technologies (ICT) and foundational concepts of electronics and communication engineering to analyze, design and develop solutions.
PSO2	Provide knowledge to create quality products for scientific and business applications by incorporating novel ideas and best practices.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

22EC603 – ADVANCED EMBEDDED SYSTEMS LABORATORY

LIST OF EXPERIMENTS

Sl. No.	Name of the Experiments
1.	CONFIGURATION AND INTEGRATION OF QNX MOMENTCS IDE WITH TARGET MACHINE
2.	IMPLEMENTATION OF MULTI-PROCESS CREATION USING SPAWN
3.	IMPLEMENTATION OF MULTI-THREAD CREATION WITH FIFO SCHEDULING ALGORITHM
4.	IMPLEMENTATION OF MULTI-THREAD CREATION WITH ROUND ROBIN SCHEDULING ALGORITHM
5.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING MUTEXES
6.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING SEMAPHORES
7.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING CONDVARS
8.	A. ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (UNNAMED) USING MESSGAE PASSING B. ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (UNNAMED) USING MESSGAE PASSING
9.	ASYNCHRONOUS NOTIFICATION HANDLING USING QNX PULSES
10.	HIGH-SPEED DATA EXCHANGE USING SHARED MEMORY AND SHARED OBJECTS
11.	INTERRUPT SERVICE ROUTINE (ISR) IMPLEMENTATION FOR EXTERNAL HARDWARE EVENTS
12.	DESIGNING A RESOURCE MANAGER FOR QNX TARGET DEVICE
13.	BUILDING AND DEPLOYING QNX BOOT/OS IMAGES

INDEX PAGE

Expt. No.	Date	Name of the Experiment	Page. No.	Marks	Signature
				TOTAL	
				AVERAGE	

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

22EC603 – ADVANCED EMBEDDED SYSTEMS LABORATORY

S.NO	NAME OF THE EXPERIMENT	COs	PO's	PEOs
1.	CONFIGURATION AND INTEGRATION OF QNX MOMENTCS IDE WITH TARGET MACHINE	CO2	1,2,3,4,5	2 & 3
2.	IMPLEMENTATION OF MULTI-PROCESS CREATION USING SPAWN	CO2	1,2,3,4,5	2 & 3
3.	IMPLEMENTATION OF MULTI-THREAD CREATION WITH FIFO SCHEDULING ALGORITHM	CO3	1,2,3,4,5	2 & 3
4.	IMPLEMENTATION OF MULTI-THREAD CREATION WITH ROUND ROBIN SCHEDULING ALGORITHM	CO3	1,2,3,4,5	2 & 3
5.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING MUTEXES	CO3	1,2,3,4,5	2 & 3
6.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING SEMAPHORES	CO3	1,2,3,4,5	2 & 3
7.	IMPLEMENTATION OF TASK SYNCHRONIZATION USING CONDVARS	CO3	1,2,3,4,5	2 & 3
8.	A. ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (UNNAMED) USING MESSGAE PASSING B. ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (UNNAMED) USING MESSGAE PASSING	CO4	1,2,3,4,5	2 & 3
9.	ASYNCHRONOUS NOTIFICATION HANDLING USING QNX PULSES	CO4	1,2,3,4,5	2 & 3
10.	HIGH-SPEED DATA EXCHANGE USING SHARED MEMORY AND SHARED OBJECTS	CO4	1,2,3,4,5	2 & 3
11.	INTERRUPT SERVICE ROUTINE (ISR) IMPLEMENTATION FOR EXTERNAL HARDWARE EVENTS	CO5	1,2,3,4,5	2 & 3
12.	DESIGNING A RESOURCE MANAGER FOR QNX TARGET DEVICE	CO5	1,2,3,4,5	2 & 3
13.	BUILDING AND DEPLOYING QNX BOOT/OS IMAGES	CO5	1,2,3,4,5	2 & 3

Department of Electronics and Communication Engineering
Laboratory Rubrics
AY 2025-26 (EVEN)

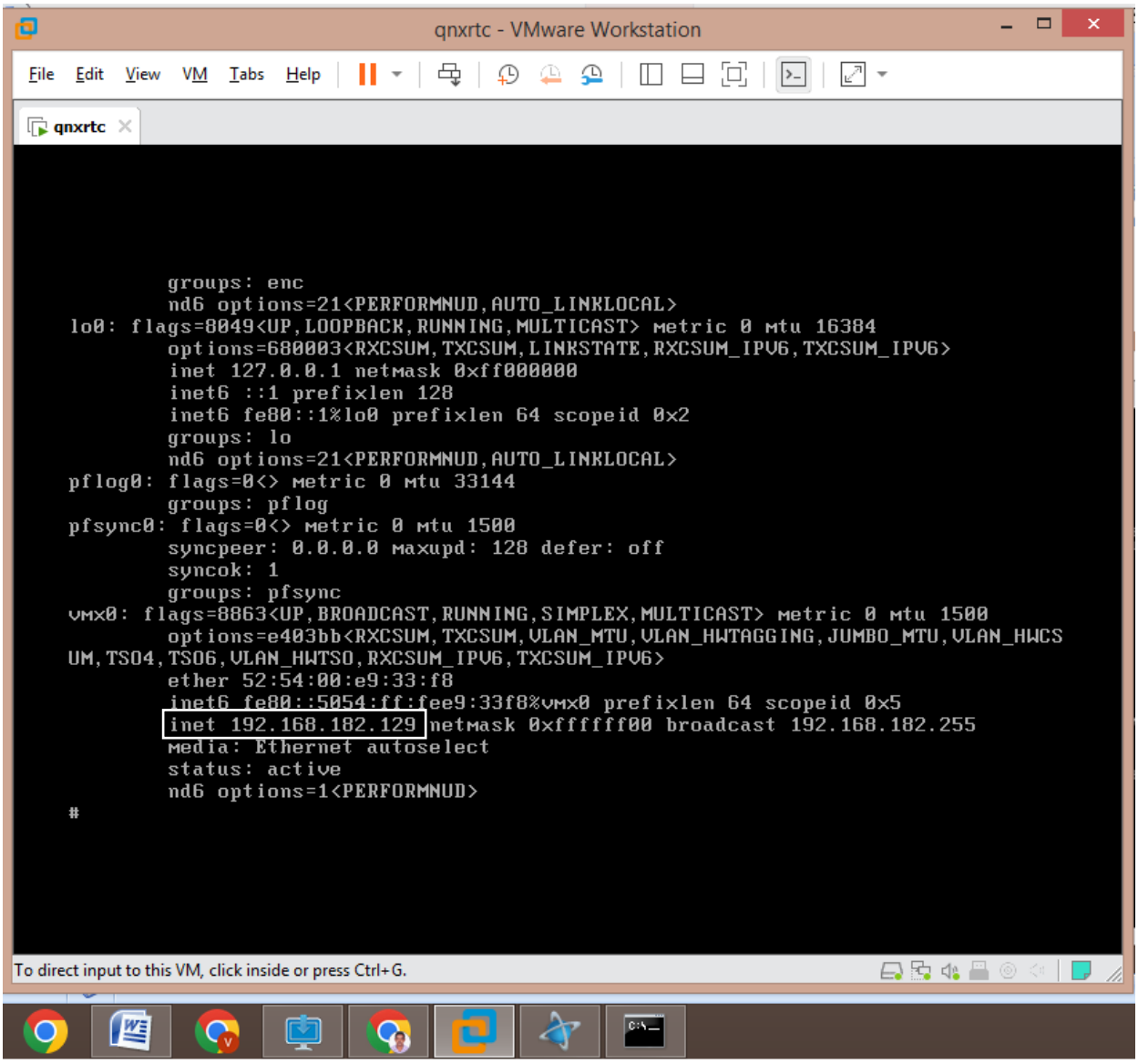
Register Number				**Course Code/Title**	**22EC603 ADVANCED EMBEDDED SYSTEMS LABORATORY**	
Name of the Student				**Name of the Evaluator**		
Criterion	**Weightage**	**Excellent (36-40)**	**Very Good (26-35)**	**Acceptable (16-25)**	**Considerable (6-15)**	**Below Expectations (0-5)**
Completion of Observation	40	Submission of lab observation in proper time with proper documentation	Submission of Lab Record in proper time with minimal errors in documentation	Late submission of Observation and with minimal errors in documentation	Late submission of Observation with major error in documentation	Late Submission of Observation
Lab Participation & Capability of doing Experiment	20	**Excellent (16-20)**	**Very Good (13-15)**	**Acceptable (9-12)**	**Considerable (5-8)**	**Below Expectations (0-4)**
		Demonstrates an accurate understanding of the experiment objectives and procedures	Demonstrates good understanding of the experiment objectives and procedures	Demonstrates an understanding of the experiment objectives and procedures	Demonstrates minimal understanding of the experiment objectives and procedures	Demonstrates nil understanding of the experiment objectives and procedures
		Experiment is done precisely based on the question	Some part of the experiment is done accurately	Some part of the experiment is done with faculty guidance	Major part of the experiment is done with faculty guidance	Experiment is done under faculty guidance
Result (Validation of Output & Overall Completion of experiments)	20	Represent all data appropriately. Finding of results are clear	Partly outputs are found	Minor parts of output are found	Major part of output not found	Wrong output/no output found
		100% target has been completed	75% target has been completed	50% target has been completed	25% target has been completed	Less than 25% target has been completed
Viva Voce	20	Demonstrate deep knowledge. Answered the questions with explanation and elaboration in viva	Adequate knowledge on most topics in viva	Answered the questions and able to elaborate in viva	Superficial knowledge of topic in viva	Answered very few questions in viva
TOTAL MARKS						

Department of Electronics and Communication Engineering
Laboratory Rubrics AY
2025-26 (EVEN) Marks
Statement

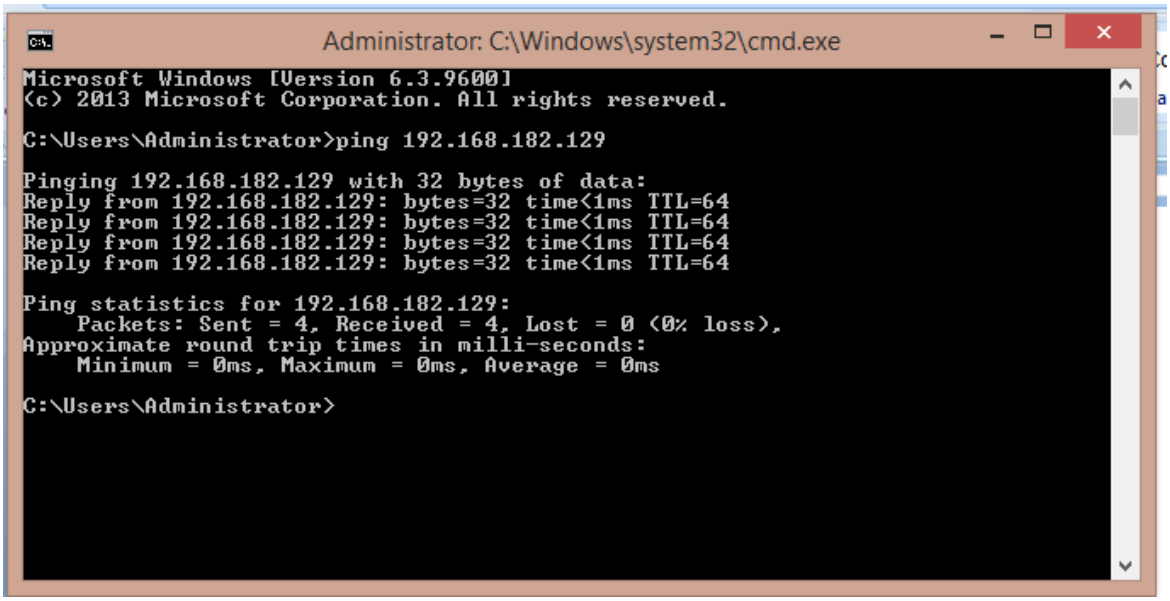
Date													
Criterion	**Exp No 1**	**Exp No 2**	**Exp No 3**	**Exp No 4**	**Exp No 5**	**Exp No 6**	**Exp No 7**	**Exp No 8**	**Exp No 9**	**Exp No 10**	**Exp No 11**	**Exp No 12**	**Exp No 13**
Completion of Observation													
Lab Participation & Capability of doing Experiment													
Result (Validation of Output & Overall													
Viva Voce													
Total Marks (out of 100)													

Signature of the Faculty – In-Charge with date

Showing IP Address after running the command "ifconfig" in target machine terminal:



Verifying the connection between Momentics IDE & Target by ping command:



Expt. No.: 1 **CONFIGURATION AND INTEGRATION OF QNX MOMENTCS IDE WITH**
Date: **TARGET MACHINE**

Aim:

To configure the QNX Momentics IDE on a host development PC and establish a communication link with a target machine (X86 Virtual Machine / Raspberry Pi 5) to enable application deployment and debugging.

Hardware Requirements:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Procedure:

Configuration of QNX Momentics IDE and Target:

Step 1: Open QNX Momentics IDE 8.0 and set workspace directory.

Step 2: Click on the menu option "File" → "New" → "Others" → "Launch Target" → Choose "QNX Virtual Machine Target" → Click on "Next".

Step 3: Edit the properties of the QNX Virtual Machine as follows and click on "Finish".

Target Name: QNX_RTC
VM Platform: vmware
CPU Architecture: x86_64
IP Address: (Leave it Blank)
Extra Options: (Leave it Blank)

Step 4: Wait till all things are setting up and automatically the QNX RTOS boot on the VMware workstation.

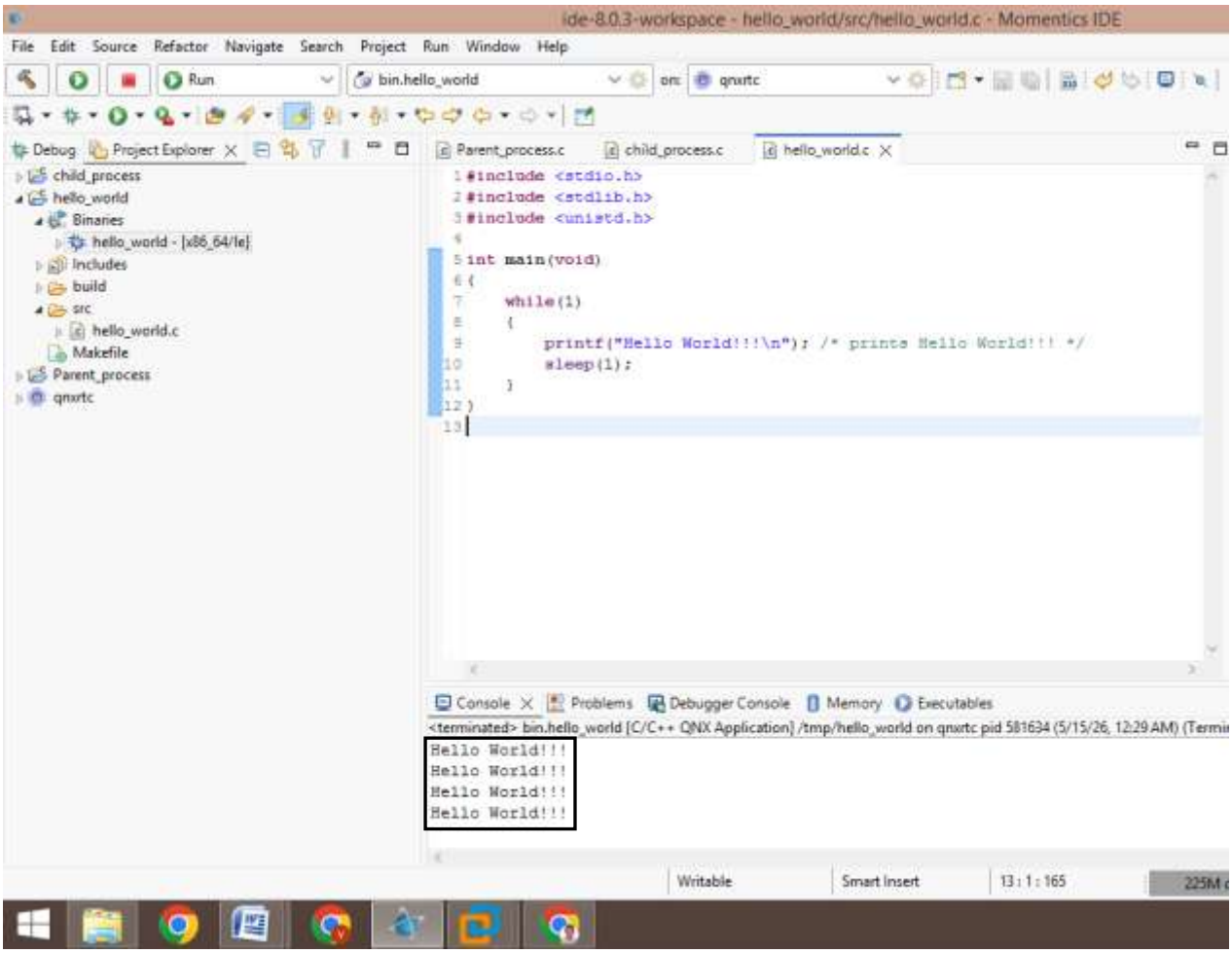
Step 5: Type the command "ifconfig" command on the terminal and note down the IP Address.

Step 6: To ensure the proper connection between QNX Momentics IDE and the target machine running on VMware Workstation, on the host PC terminal type the command "ping 192.168.xxx.xxx" and verify 0% data loss.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
	while(1)
	{
		printf("Hello World!!!\n"); /* prints Hello World!!! */
		sleep(1);
	}
}
```

Output:



Deploy a sample QNX Application:

Step 1: In the QNX Momentics IDE 8.0, "File" → "New" → "QNX Project → "QNX Executable".

Step 2: Type the project name and choose the "CPU variant" and click on "Finish".

Step 3: In the Project Explorer window select the project directory created and navigate to the folder named as "src" and find the .c source file.

Step 4: Type a sample code to print "Hello World!" for every 1 second on the console.

Step 5: Right Click on the source file and click on "Build Project" to build the project.

Step 6: Under the project directory navigate to the folder named as "Binaries" and right click on the executable file created and click on "Run As" → "C/C++ QNX Applications"

Step 7: Observe the output on the console window.

Algorithm:

Step 1: Start - The execution of the program begins at the main() function.

Step 2: Initialization - The standard input/output and system utility libraries (stdio.h, stdlib.h, and unistd.h) are included to support printing and timing functions.

Step 3: Enter Infinite Loop - A while(1) loop is initiated, which will continue to execute indefinitely as long as the condition evaluates to true (non-zero).

Step 4: Print Output - The printf function displays the string "Hello World!!!" followed by a newline character on the standard output console.

Step 5: Delay Execution - The sleep(1) function is called to pause the process execution for exactly 1 second.

Step 6: Repeat - The control returns to the beginning of the while loop (Step 4).

Results:

Thus the configuration and integration of the QNX Momentics IDE 8.0 with the x86_64 Virtual Machine were successfully completed. The establishment of a stable communication link enabled the seamless deployment and execution of a C application on the Neutrino RTOS. This setup confirms the readiness of the development environment for complex real-time application debugging and system-level programming.

Source Code (parent_process):

```
#include <stdio.h>
#include <stdlib.h>
#include <process.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid;
    printf("Parent Process: Launching Child...\n");

    // Using spawnl to create a new process in a protected address space
    pid = spawnl(P_NOWAIT, "/tmp/child_process", "child_process", NULL);

    if (pid == -1) {
        perror("spawnl");
        return EXIT_FAILURE;
    }

    printf("Parent Process: Child launched with PID %d\n", pid);
    return EXIT_SUCCESS;
}
```

Source Code (child_process):

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
    // Get the Process ID of the current (child) process
    pid_t my_pid = getpid();

    printf("----------------------------------------\n");
    printf("Child Process: Hello! My PID is [%d]\n", my_pid);
    printf("Child Process: Running in a protected address space.\n");
    printf("----------------------------------------\n");

    // Keep the process alive for a few seconds so you can see it in the
    // QNX System Information target view
    sleep(5);

    printf("Child Process [%d]: Task complete. Exiting...\n", my_pid);

    return EXIT_SUCCESS;
}
```

Expt. No.: 2 **IMPLEMENTATION OF MULTI-PROCESS CREATION USING SPAWN**

Date:

Aim:

To implement and verify multi-process creation in the QNX Neutrino RTOS using the spawn() system call, demonstrating the execution of independent parent and child processes within protected address spaces.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm (parent_process):

Step 1: Start - Execution begins at the main() function.

Step 2: Spawn Call - The spawnl() system call is invoked with the P_NOWAIT flag to launch the child binary independently.

Step 3: Error Handling - If the returned PID is -1, print an error and terminate; Else, display the child's PID.

Step 4: End - The parent process finishes its task and exits

Algorithm (child_process):

Step 1: Start - Execution is triggered by the parent's spawn request.

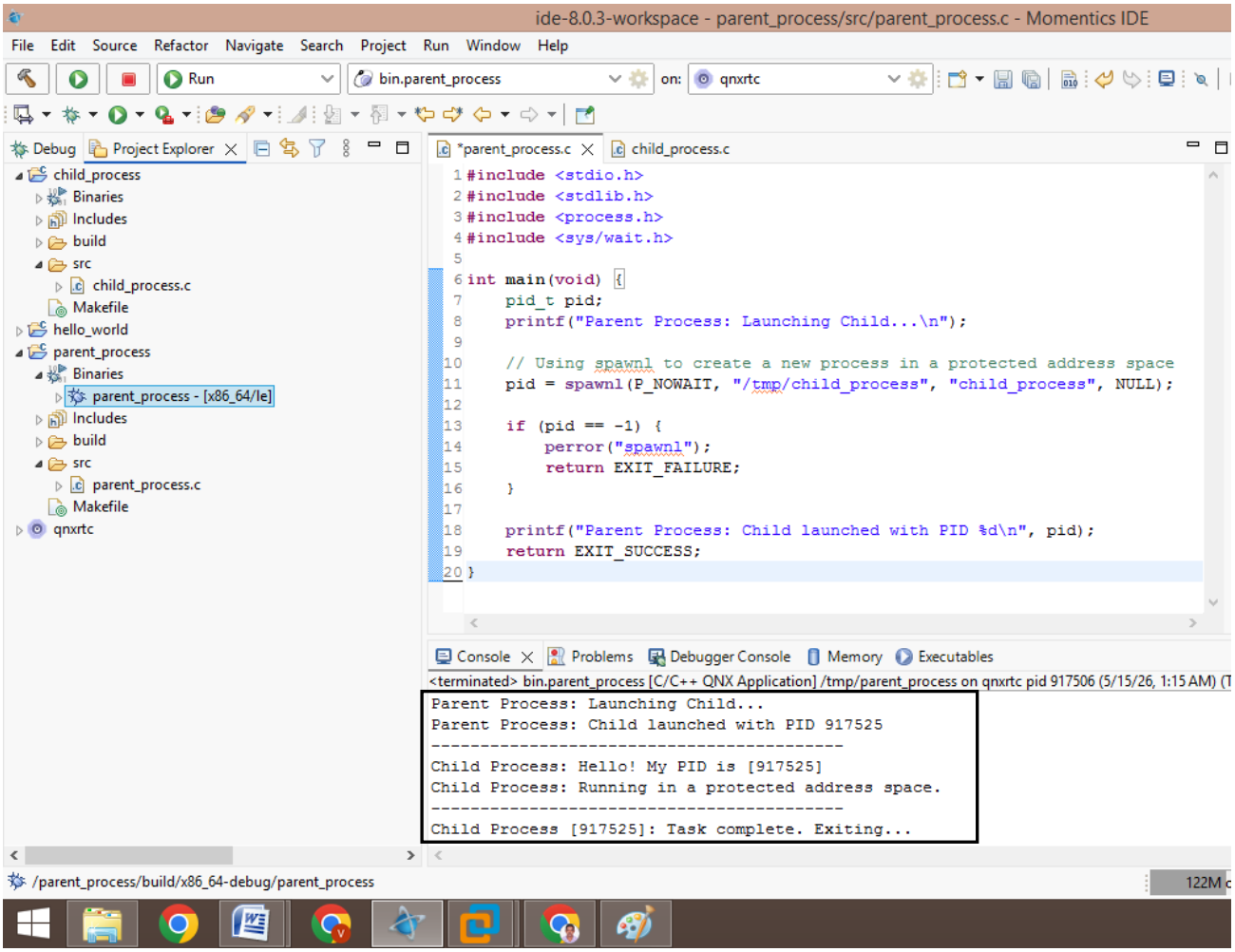
Step 2: Identification - The process retrieves its own PID using getpid() to confirm system entry.

Step 3: Execution - Print a confirmation message to the console.

Step 4: State Delay - Execute sleep(5) to transition to a blocked state, simulating real-time task behavior.

Step 5: Termination - The child process returns EXIT_SUCCESS to the microkernel.

Output:


ide-8.0.3-workspace - parent_process/src/parent_process.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Run
bin.parent_process
on:
qnxrtc
Debug
Project Explorer
child_process
Binaries
Includes
build
src
child_process.c
Makefile
hello_world
parent_process
Binaries
parent_process - [x86_64/le]
Includes
build
src
parent_process.c
Makefile
qnxrtc
*parent_process.c
child_process.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <process.h>
4 #include <sys/wait.h>
5
6 int main(void) {
7 pid_t pid;
8 printf("Parent Process: Launching Child...\n");
9
10 // Using spawnl to create a new process in a protected address space
11 pid = spawnl(P_NOWAIT, "/tmp/child_process", "child_process", NULL);
12
13 if (pid == -1) {
14 perror("spawnl");
15 return EXIT_FAILURE;
16 }
17
18 printf("Parent Process: Child launched with PID %d\n", pid);
19 return EXIT_SUCCESS;
20 }
Console
Problems
Debugger Console
Memory
Executables
<terminated> bin.parent_process [C/C++ QNX Application] /tmp/parent_process on qnxrtc pid 917506 (5/15/26, 1:15 AM)
Parent Process: Launching Child...
Parent Process: Child launched with PID 917525
Child Process: Hello! My PID is [917525]
Child Process: Running in a protected address space.
Child Process [917525]: Task complete. Exiting...
/parent_process/build/x86_64-debug/parent_process
122M

Procedure:

Step 1: Create two separate QNX Executable projects named "parent_process" and "child_process".

Step 2: Implement the spawnl() function in the parent source file to launch the child process binary.

Step 3: Implement a standard output and delay sequence in the child source file to confirm execution.

Step 4: Right-click both projects and select "Build Project" to generate the target-specific binaries.

Step 5: Right click on the executable file "parent_process" located under "Binaries" folder and select "Run As" → "C/C++ QNX Application Configuartion", navigate to the "Upload" tab and add the child_process binary to the file transfer list to ensure both are deployed to the target directory (e.g., /tmp/).

Step 6: Run the Parent_Process as a "C/C++ QNX Application" and monitor the IDE console for the interleaved output of both processes.

Results:

The implementation of multi-process creation using spawnl() was successfully verified on the QNX Neutrino RTOS. The output confirmed that the parent could successfully initiate a child process, which then executed independently within its own protected memory space, adhering to POSIX standards for system application development.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>

int global_counter = 0;

typedef struct
{
    int start;
    int end;
    int thread_id;
} thread_data_t;

void* count_function(void* arg)
{
    thread_data_t* data = (thread_data_t*)arg;
    for (int i = data->start; i <= data->end; i++)
    {
        global_counter = i;
    }
    printf("Thread %d finished. Current global_counter: %d\n", data-
>thread_id, global_counter);
    return NULL;
}

int main()
{
    pthread_t threads[4];
    thread_data_t t_data[4];
    pthread_attr_t attr;
    struct sched_param param;
    int ranges[4][2] = {{0, 250}, {251, 500}, {501, 750}, {751, 1000}};

    pthread_attr_init(&attr);
    pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
    param.sched_priority = 10;
    pthread_attr_setschedparam(&attr, &param);

    for (int i = 0; i < 4; i++)
    {
        t_data[i].start = ranges[i][0];
        t_data[i].end = ranges[i][1];
        t_data[i].thread_id = i + 1;
        pthread_create(&threads[i], &attr, count_function, &t_data[i]);
    }

    for (int i = 0; i < 4; i++)
    {
      pthread_join(threads[i], NULL);
    }

    printf("Final Global Counter Value: %d\n", global_counter);
    return EXIT_SUCCESS;
}
```

Expt. No.: 3 **IMPLEMENTATION OF MULTI-THREAD CREATION WITH FIFO**
Date: **SCHEDULING ALGORITHM**

Aim:

To implement multi-threaded programming in the QNX Neutrino RTOS using POSIX threads (pthreads) and verify the non-preemptive behavior of the First-In-First-Out (FIFO) scheduling policy using a shared global variable.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - The program begins at the main() function.

Step 2: Initialization - Include standard libraries (stdio.h, pthread.h, sched.h) and initialize thread attributes for SCHED_FIFO.

Step 3: Define Ranges - Divide the task into four ranges: 0-250, 251-500, 501-750, and 751-1000.

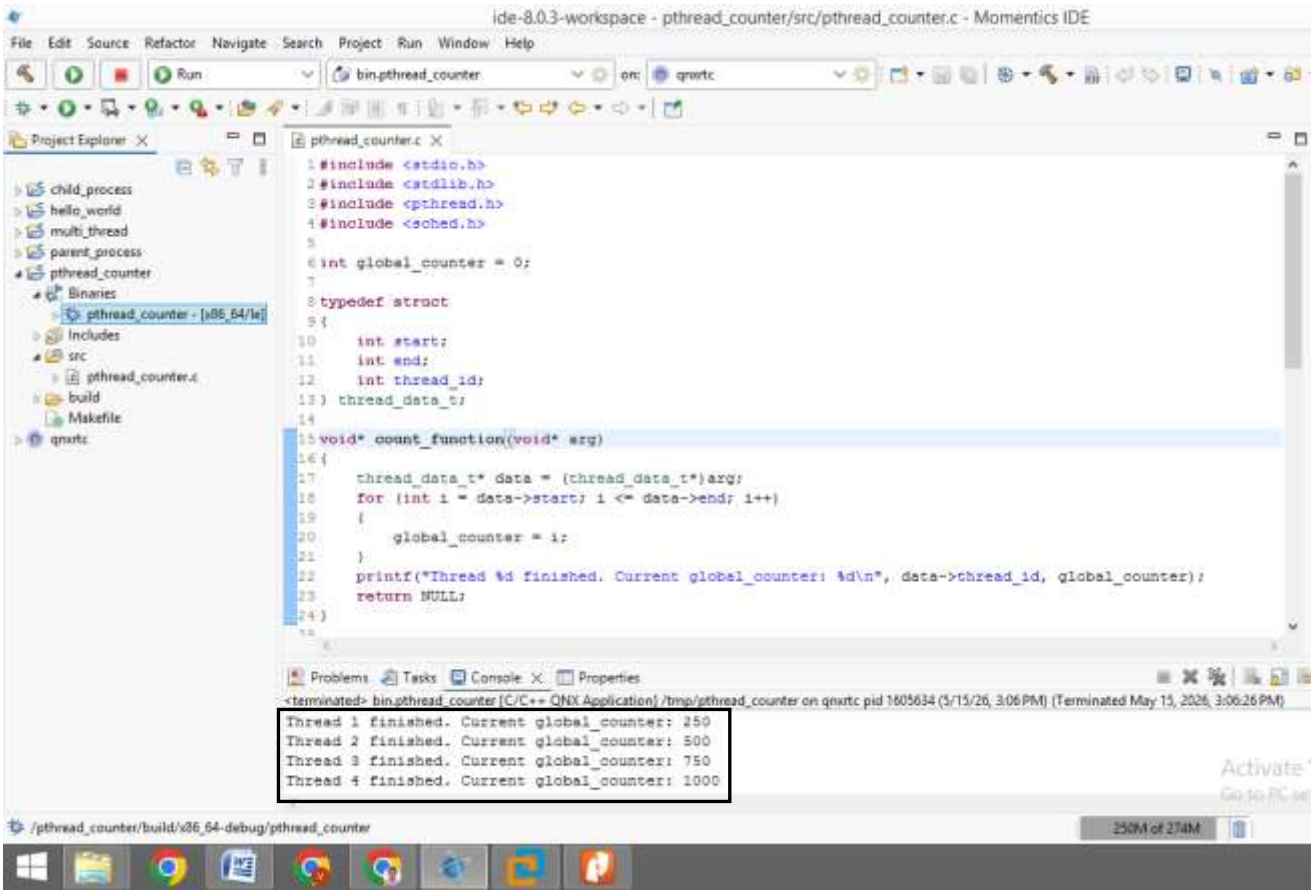
Step 4: Thread Creation - Spawn four threads using pthread_create() with the FIFO attribute.

Step 5: Due to FIFO scheduling, each thread runs its loop to completion without being preempted by other equal-priority threads.

Step 6: The main thread waits for all child threads to finish and prints the final global count.

Step 7: End - Process terminates.

Output:


ide-8.0.3-workspace - pthread_counter/src/pthread_counter.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
child_process
hello_world
multi_thread
parent_process
pthread_counter
Binaries
pthread_counter - [x86_64/le]
Includes
src
pthread_counter.c
build
Makefile
qnxrtc
pthread_counter.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
int global_counter = 0;
typedef struct
{
int start;
int end;
int thread_id;
} thread_data_t;
void* count_function(void* arg)
{
thread_data_t* data = (thread_data_t*)arg;
for (int i = data->start; i <= data->end; i++)
{
global_counter = i;
}
printf("Thread %d finished. Current global_counter: %d\n", data->thread_id, global_counter);
return NULL;
}
Problems Tasks Console Properties
<terminated> bin.pthread_counter [C/C++ QNX Application] /tmp/pthread_counter on qnxrtc pid 1605634 (5/15/26, 3:06 PM) (Terminated May 15, 2026, 3:06:26 PM)
Thread 1 finished. Current global_counter: 250
Thread 2 finished. Current global_counter: 500
Thread 3 finished. Current global_counter: 750
Thread 4 finished. Current global_counter: 1000
/pthread_counter/build/x86_64-debug/pthread_counter
250M of 274M

Procedure:

Step 1: Create a new QNX Executable project in the Momentics IDE.

Step 2: Define a global counter and a structure to pass range parameters (0-250, 251-500, 501-750 & 751-1000) to four independent threads (thread1, thread2, thread3 & thread4) respectively.

Step 3: Initialize thread attributes and use pthread_attr_setschedpolicy() to set the policy to SCHED_FIFO.

Step 4: Set a consistent priority level for all threads to observe equal-priority scheduling behavior.

Step 5: Build the project and run it as a C/C++ QNX Application.

Step 6: Monitor the console output to verify that threads execute their full ranges sequentially without preemption.

Results:

The implementation of multi-thread creation with FIFO scheduling was successfully completed. The non-preemptive nature of the FIFO policy ensured that each threads processed its assigned range sequentially, demonstrating the microkernel's ability to maintain process integrity in a real-time environment.

Source Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>

int global_rr_count = 0;

void* rr_worker(void* arg)
{
    int id = *((int*)arg);
    while (global_rr_count < 20)
    {
        global_rr_count++;
        printf("RR Thread %d incremented count to: %d\n", id, global_rr_count);
        for (volatile int j = 0; j < 100000; j++);
    }
    return NULL;
}

int main()
{
    pthread_t threads[4];
    int ids[4] = {1, 2, 3, 4};
    pthread_attr_t attr;
    struct sched_param param;

    pthread_attr_init(&attr);
    pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr, SCHED_RR);
    param.sched_priority = 10;
    pthread_attr_setschedparam(&attr, &param);

    for (int i = 0; i < 4; i++)
    {
        pthread_create(&threads[i], &attr, rr_worker, &ids[i]);
    }

    for (int i = 0; i < 4; i++)
    {
      pthread_join(threads[i], NULL);
    }

    printf("Final Round Robin Counter: %d\n", global_rr_count);
    return EXIT_SUCCESS;
}
```

Expt. No.: 4 IMPLEMENTATION OF MULTI-THREAD CREATION WITH ROUND ROBIN
Date: SCHEDULING ALGORITHM

Aim:

To implement multi-threaded programming in the QNX Neutrino RTOS and verify the time-slicing behavior of the Round Robin (RR) scheduling policy using shared global variable updates.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - The program begins at the main() function.

Step 2: Initialization - Include standard libraries (stdio.h, pthread.h, sched.h) and initialize thread attributes for SCHED_FIFO.

Step 3: Define Ranges - Divide the task into four ranges: 0-250, 251-500, 501-750, and 751-1000.

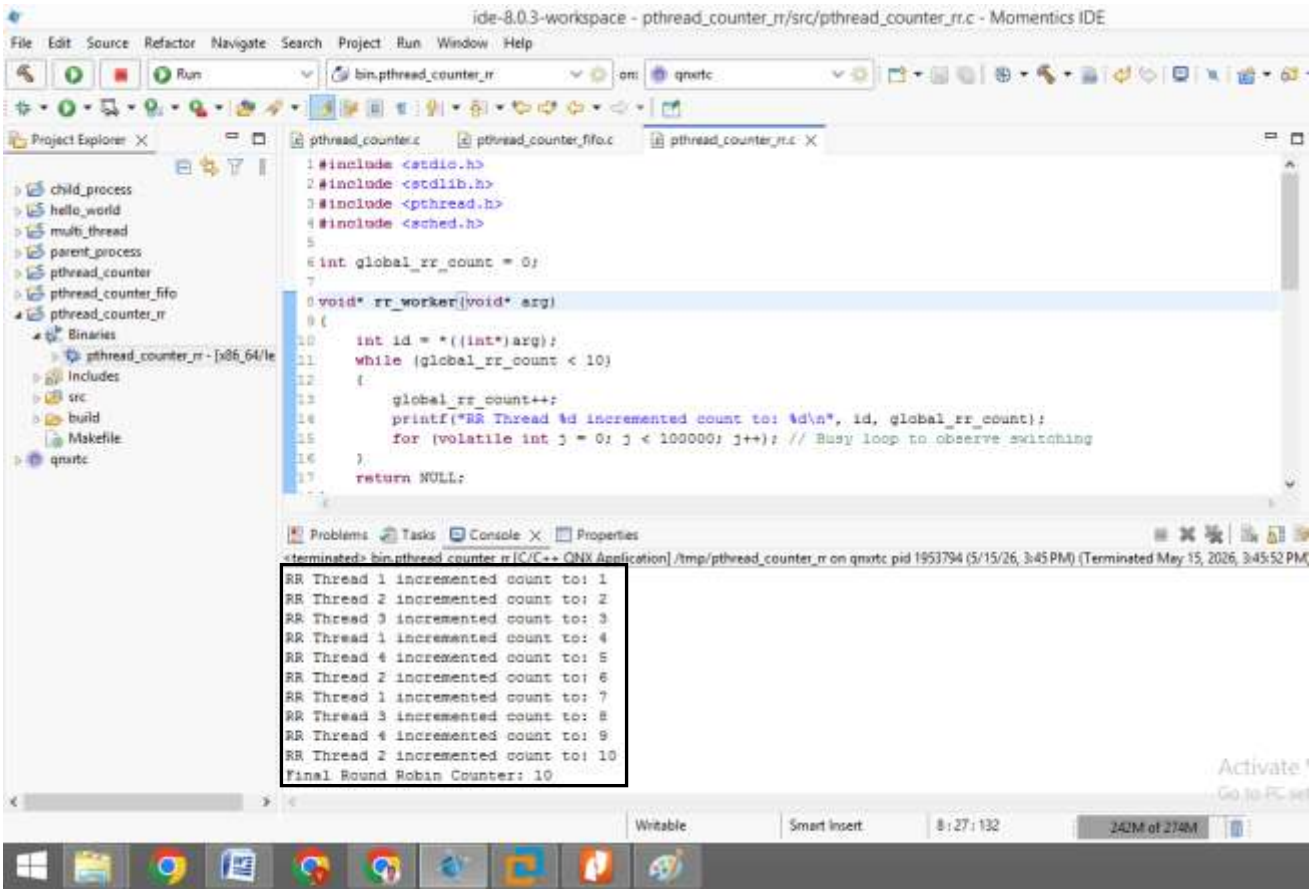
Step 4: Thread Creation - Spawn four threads using pthread_create() with the FIFO attribute.

Step 5: Due to FIFO scheduling, each thread runs its loop to completion without being preempted by other equal-priority threads.

Step 6: The main thread waits for all child threads to finish and prints the final global count.

Step 7: End - Process terminates.

Output:


ide-8.0.3-workspace - pthread_counter_rr/src/pthread_counter_rr.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
child_process
hello_world
multi_thread
parent_process
pthread_counter
pthread_counter_fifo
pthread_counter_rr
Binaries
Includes
src
build
Makefile
qnxrtc
pthread_counter.c
pthread_counter_fifo.c
pthread_counter_rr.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
int global_rr_count = 0;
void* rr_worker(void* arg)
{
int id = *((int*)arg);
while (global_rr_count < 10)
{
global_rr_count++;
printf("RR Thread %d incremented count to: %d\n", id, global_rr_count);
for (volatile int j = 0; j < 100000; j++); // Busy loop to observe switching
}
return NULL;
Problems Tasks Console Properties
RR Thread 1 incremented count to: 1
RR Thread 2 incremented count to: 2
RR Thread 3 incremented count to: 3
RR Thread 1 incremented count to: 4
RR Thread 4 incremented count to: 5
RR Thread 2 incremented count to: 6
RR Thread 1 incremented count to: 7
RR Thread 3 incremented count to: 8
RR Thread 4 incremented count to: 9
RR Thread 2 incremented count to: 10
Final Round Robin Counter: 10
Writable
Smart Insert

Procedure:

Step 1: Create a new QNX Executable project in the Momentics IDE.

Step 2: Define a global counter that all four threads will attempt to increment until a final limit of 1000 is reached.

Step 3: Initialize thread attributes and use pthread_attr_setschedpolicy() to set the policy to SCHED_RR.

Step 4: Include a printf or small busy-loop within the thread function to facilitate context switching observation. Include a printf or small busy-loop within the thread function to facilitate context switching observation.

Step 5: Build the project and run it as a C/C++ QNX Application.

Step 6: Monitor the console output to verify that each threads executes with time slicing to increment the global count till it reaches 10.

Results:

The implementation of multi-thread creation with Round Robin scheduling was successfully completed. The experiment confirmed that the Round Robin scheduling policy enables fair CPU sharing through time-slicing, as evidenced by the interleaved thread execution and balanced incrementing of the shared global counter.

Source Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
#include <unistd.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
int global_count = 0;

void* thread_func(void* arg)
{
    int id = *(int*)arg;

    for (int i = 0; i < 1; i++)
    {
        printf("Thread %d: Attempting to LOCK mutex...\n", id);

        pthread_mutex_lock(&my_mutex);

        printf("[LOCKED] Thread %d entered critical section.\n", id);
        global_count++;
        printf("Thread %d: Updated global count to %d\n", id, global_count);

        printf("[UNLOCKING] Thread %d is leaving critical section.\n", id);
        pthread_mutex_unlock(&my_mutex);

        usleep(100000);
    }
    return NULL;
}

int main()
{
    pthread_t threads[3];
    int thread_ids[3];
    struct sched_param param;

    param.sched_priority = 10;
    if (sched_setscheduler(0, SCHED_RR, &param) == -1)
    {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }

    printf("Starting Mutex Experiment with 3 Threads (RR Scheduling)...\n");

    for (int i = 0; i < 3; i++)
    {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &thread_ids[i]);
    }

    for (int i = 0; i < 3; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final global count Value: %d\n", global_count);
    return 0;
}
```

Expt. No.: 5 **IMPLEMENTATION OF TASK SYNCHRONIZATION USING MUTEX**

Date:

Aim:

To implement task synchronization using a Mutex in QNX Neutrino RTOS to protect a shared global variable from race conditions using Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - Initialize the program and set the system scheduler to Round Robin (SCHED_RR).

Step 2: Resource Allocation - Declare a global variable global_count and a Mutex object.

Step 3: Thread Spawning - Create 3 concurrent threads, each passing a unique ID to the task function.

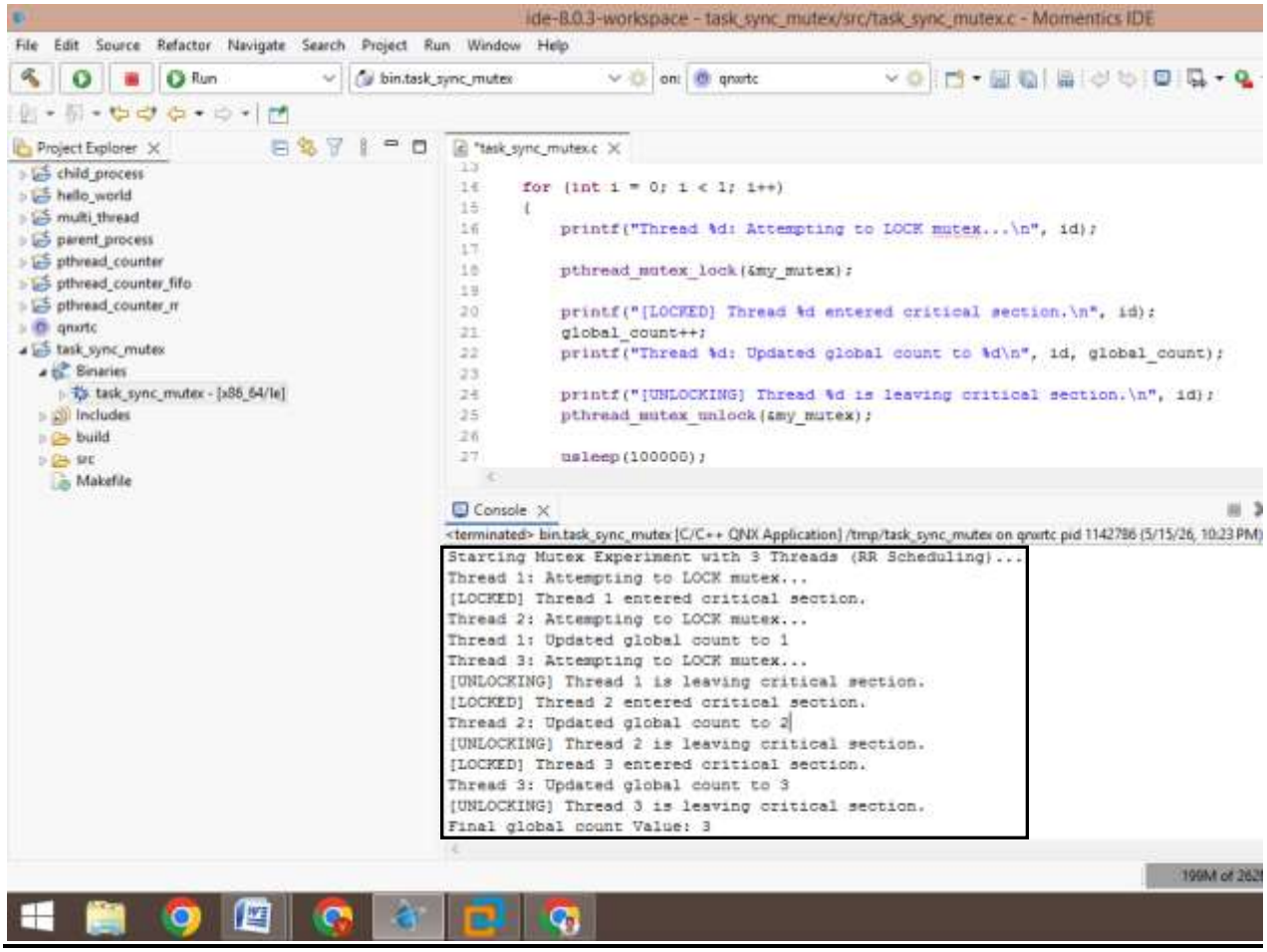
Step 4: Lock Acquisition - Each thread attempts to acquire the Mutex. If the Mutex is held by another thread, the current thread blocks.

Step 5: Data Modification - Once the lock is acquired, the thread increments global_count and prints the status.

Step 6: Release - The thread releases the Mutex, allowing the next thread in the RR queue to acquire it.

Step 7: Synchronization - The main function uses pthread_join to wait for all threads to finish before printing the final result.

Output:


ide-8.0.3-workspace - task_sync_mutex/src/task_sync_mutex.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Run
bin.task_sync_mutex
on:
qnxrtc
Project Explorer
child_process
hello_world
multi_thread
parent_process
pthread_counter
pthread_counter_fifo
pthread_counter_rr
qnxrtc
task_sync_mutex
Binaries
task_sync_mutex - [x86_64/le]
Includes
build
src
Makefile
*task_sync_mutex.c
for (int i = 0; i < 1; i++)
{
printf("Thread %d: Attempting to LOCK mutex...\n", id);
pthread_mutex_lock(&my_mutex);
printf("[LOCKED] Thread %d entered critical section.\n", id);
global_count++;
printf("Thread %d: Updated global count to %d\n", id, global_count);
printf("[UNLOCKING] Thread %d is leaving critical section.\n", id);
pthread_mutex_unlock(&my_mutex);
usleep(100000);
Console
<terminated> bin.task_sync_mutex [C/C++ QNX Application] /tmp/task_sync_mutex on qnxrtc pid 1142786 (5/15/26, 10:23 PM)
Starting Mutex Experiment with 3 Threads (RR Scheduling)...
Thread 1: Attempting to LOCK mutex...
[LOCKED] Thread 1 entered critical section.
Thread 2: Attempting to LOCK mutex...
Thread 1: Updated global count to 1
Thread 3: Attempting to LOCK mutex...
[UNLOCKING] Thread 1 is leaving critical section.
[LOCKED] Thread 2 entered critical section.
Thread 2: Updated global count to 2
[UNLOCKING] Thread 2 is leaving critical section.
[LOCKED] Thread 3 entered critical section.
Thread 3: Updated global count to 3
[UNLOCKING] Thread 3 is leaving critical section.
Final global count Value: 3

Procedure:

Step 1: Create a new QNX Executable project in the Momentics IDE.

Step 2: Use sched_setscheduler() to set the policy to SCHED_RR with a priority of 10 to ensure equal time-slicing among threads.

Step 3: Define a global pthread_mutex_t and initialize it with PTHREAD_MUTEX_INITIALIZER.

Step 4: Spawn 3 worker threads using pthread_create().

Step 5: Inside the thread function, use pthread_mutex_lock() before accessing the global_count and pthread_mutex_unlock() immediately after access.

Step 6: Include printf statements to track the "Attempting to LOCK," "LOCKED," and "UNLOCKING" states.

Step 7: Build the project and run it as a C/C++ QNX Application.

Step 8: Monitor the console output to verify that each threads locks mutex object, update and unlock the mutex object.

Results:

The Mutex successfully synchronized the 3 threads, ensuring that the global_count was updated sequentially without interference. The console output confirmed that only one thread held the "LOCKED" status at any given time, demonstrating effective mutual exclusion under QNX RTOS.

Source Code:

```c
#include <stdio.h>
#include <semaphore.h>
#include <pthread.h>
#include <unistd.h>
#include <sched.h>

sem_t slot_limit;
int global_count = 0;

void* worker(void* arg)
{
    int id = *(int*)arg;

    printf("Thread %d: Waiting for an available slot...\n", id);
    sem_wait(&slot_limit);

    printf("Thread %d: ENTERED slot. Accessing resource...\n", id);
    global_count++;

    printf("Thread %d: Incrementing global count to: %d\n", id, global_count);
    sleep(1); // Simulating work

    printf("Thread %d: Finished. LEAVING slot.\n", id);
    sem_post(&slot_limit);

    return NULL;
}

int main()
{
    pthread_t threads[3];
    int ids[3];
    struct sched_param param;

    param.sched_priority = 10;
    sched_setscheduler(0, SCHED_RR, &param);

    sem_init(&slot_limit, 0, 2);

    printf("Starting Semaphore Experiment (2 Slots, 3 Threads)...\n");

    for (int i = 0; i < 3; i++)
    {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < 3; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final Global Count: %d\n", global_count);
    sem_destroy(&slot_limit);

    return 0;
}
```

Expt. No.: 6 IMPLEMENTATION OF TASK SYNCHRONIZATION USING SEMAPHORES

Date:

Aim:

To implement task synchronization using a Semaphore in QNX Neutrino RTOS to manage access to a limited number of shared resource slots among multiple threads using Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - Begin the program and set the scheduling policy to Round Robin (SCHED_RR).

Step 2: Semaphore Initialization - Initialize a semaphore with an initial count of 2, allowing only two threads to proceed at once.

Step 3: Parallel Execution - Spawn 3 concurrent threads; each thread calls the worker function.

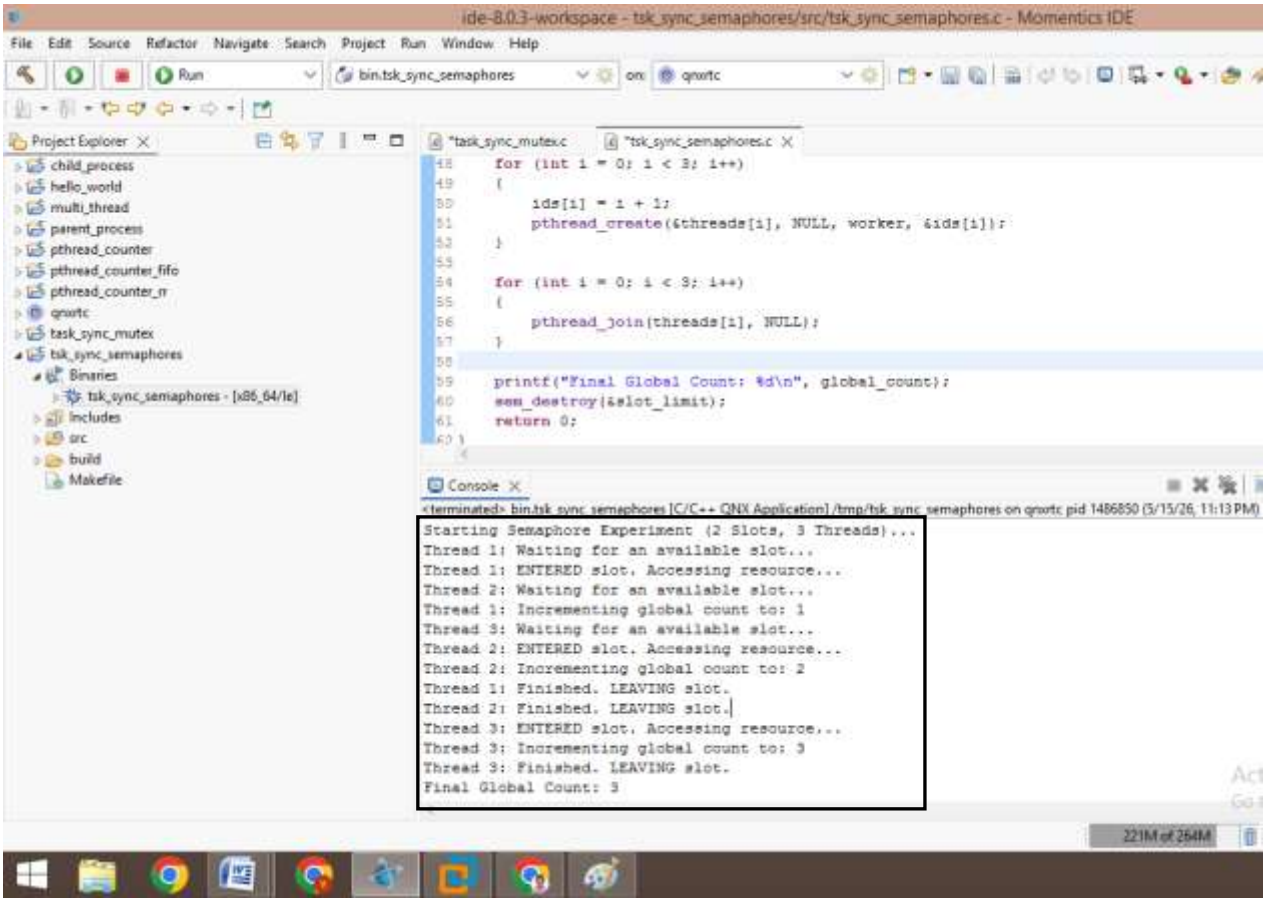
Step 4: Acquire Permit - Each thread executes sem_wait(). If the semaphore count is greater than 0, it decrements the count and enters the slot. Otherwise, it waits in the queue.

Step 5: Update Shared Resource - The thread increments the global_count and prints the progress status.

Step 6: Release Permit - After completing the work, the thread executes sem_post(), incrementing the semaphore count and allowing waiting threads to enter.

Step 7: Join & Cleanup - The main thread waits for all workers to finish using pthread_join() and destroys the semaphore.

Output:


ide-8.0.3-workspace - tsk_sync_semaphores/src/tsk_sync_semaphores.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
child_process
hello_world
multi_thread
parent_process
pthread_counter
pthread_counter_fifo
pthread_counter_rr
qnxrtc
task_sync_mutex
tsk_sync_semaphores
Binaries
tsk_sync_semaphores - [x86_64/le]
Includes
src
build
Makefile
*task_sync_mutex.c
*tsk_sync_semaphores.c
for (int i = 0; i < 3; i++)
{
ids[i] = i + 1;
pthread_create(&threads[i], NULL, worker, &ids[i]);
}
for (int i = 0; i < 3; i++)
{
pthread_join(threads[i], NULL);
}
printf("Final Global Count: %d\n", global_count);
sem_destroy(&slot_limit);
return 0;
Console
Starting Semaphore Experiment (2 Slots, 3 Threads)...
Thread 1: Waiting for an available slot...
Thread 1: ENTERED slot. Accessing resource...
Thread 2: Waiting for an available slot...
Thread 1: Incrementing global count to: 1
Thread 3: Waiting for an available slot...
Thread 2: ENTERED slot. Accessing resource...
Thread 2: Incrementing global count to: 2
Thread 1: Finished. LEAVING slot.
Thread 2: Finished. LEAVING slot.
Thread 3: ENTERED slot. Accessing resource...
Thread 3: Incrementing global count to: 3
Thread 3: Finished. LEAVING slot.
Final Global Count: 3

Procedure:

Step 1: Create a new QNX Executable project in the Momentics IDE.

Step 2: Use sched_setscheduler() to set the process to SCHED_RR with a priority level of 10 to ensure equal time-slicing for all threads.

Step 3: Define a global sem_t and initialize it with a value of 2 using sem_init(), representing two available resource slots.

Step 4: Create 3 threads using pthread_create() that attempt to enter the slots simultaneously.

Step 5: Use sem_wait() (P operation) to acquire a slot and sem_post() (V operation) to release it.

Step 6: Increment a shared global_count within the protected region and include a sleep(1) to simulate a time-intensive task.

Step 7: Build the project and run it as a C/C++ QNX Application.

Step 8: Monitor the console output to verify that each threads increment the global count with synchronization using semaphore with count 2.

Results:

The experiment confirmed successful task synchronization using a semaphore. Observations in the console showed that while 3 threads were active, only 2 could occupy a resource slot at any given time, preventing race conditions and managing resource distribution effectively under the Neutrino RTOS.

Source Code:

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <sched.h>
#include <stdlib.h>

pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int system_ready = 0;
int global_count = 0;

void* worker_thread(void* arg)
{
    int id = *(int*)arg;
    pthread_mutex_lock(&mtx);

    while (system_ready == 0)
    {
        printf("Thread %d: Waiting for system_ready signal...\n", id);
        pthread_cond_wait(&cv, &mtx);
    }

    printf("Thread %d: Signal received! Performing task...\n", id);
    global_count++;
    printf("Thread %d: Incrementing global count to: %d\n", id, global_count);
    pthread_mutex_unlock(&mtx);

    return NULL;
}

int main()
{
    pthread_t threads[4];
    int ids[4];
    struct sched_param param;
    param.sched_priority = 10;

    if (sched_setscheduler(0, SCHED_RR, &param) == -1)
    {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }
    printf("Starting Condvar Experiment (4 Threads, RR Scheduling)...\n");
    for (int i = 0; i < 4; i++)
    {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, worker_thread, &ids[i]);
    }
    sleep(2);

    printf("Main: System is now READY. Signaling all threads...\n");
    pthread_mutex_lock(&mtx);
    system_ready = 1;
    pthread_cond_broadcast(&cv); // Wake up all 4 waiting threads
    pthread_mutex_unlock(&mtx);

    for (int i = 0; i < 4; i++)
    {
      Pthread_join(threads[i], NULL);
    }
    printf("Final Global Count: %d\n", global_count);
    return 0;
}
```

Expt. No.: 7 IMPLEMENTATION OF TASK SYNCHRONIZATION USING CONDVARS

Date:

Aim:

To implement task synchronization using **Condition Variables (Condvars)** in QNX Neutrino RTOS to coordinate the simultaneous start of multiple threads and protect a shared counter using **Round Robin (RR)** scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - Program begins at main() and sets the scheduler to Round Robin (SCHED_RR).

Step 2: Initial State - 4 threads are created and immediately enter a waiting state via pthread_cond_wait() because the system_ready flag is 0.

Step 3: Wait State - While blocked, the threads consume no CPU cycles.

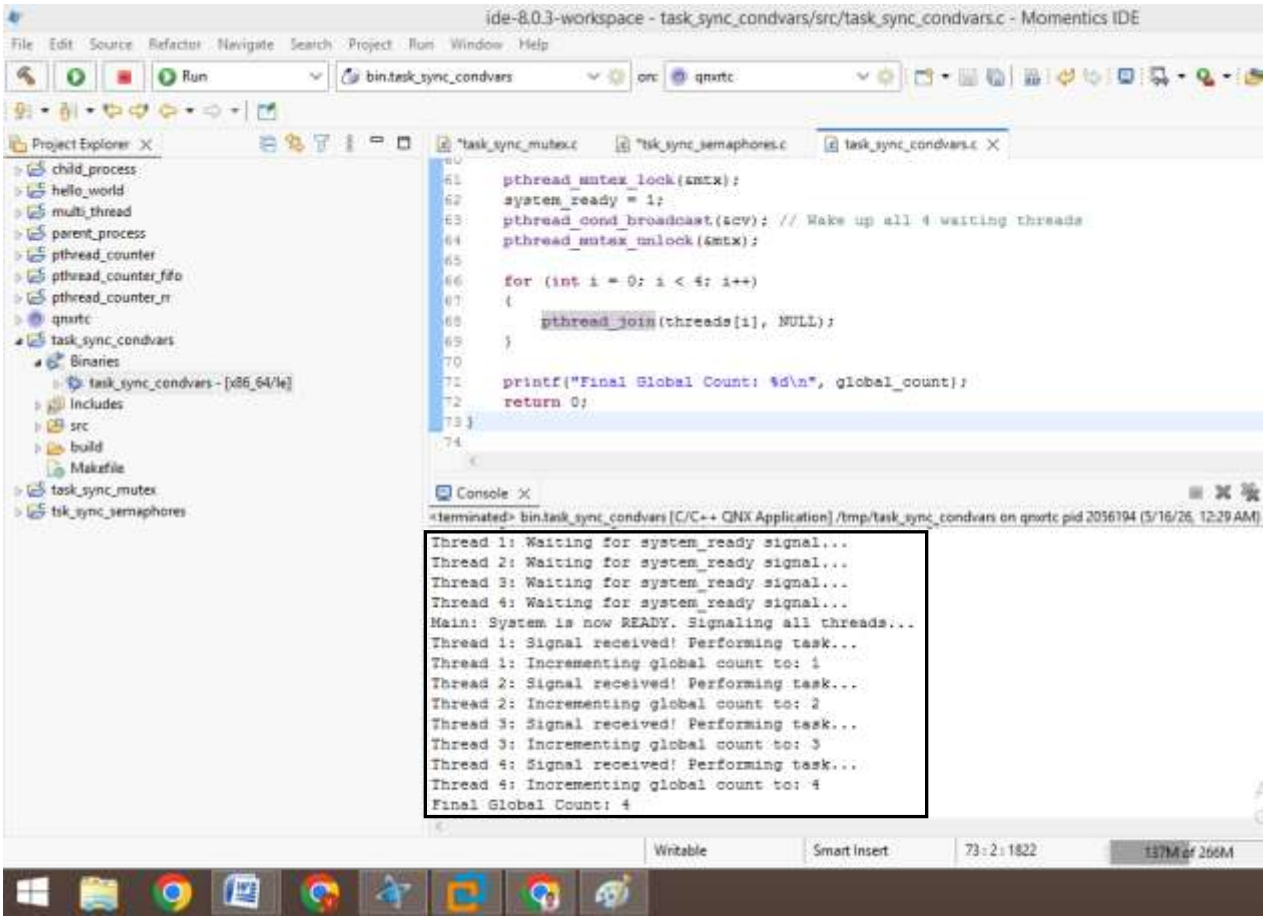
Step 4: Trigger - The main thread waits for 2 seconds, then acquires the mutex to set system_ready = 1.

Step 5: Notification - pthread_cond_broadcast() is called, moving all 4 threads from the "Blocked" queue to the "Ready" queue.

Step 6: Task - Each thread, in turn, re-acquires the mutex, increments the global_count, and prints its progress status.

Step 7: Cleanup - The main thread joins all workers and prints the final count before terminating.

Output:


ide-8.0.3-workspace - task_sync_condvars/src/task_sync_condvars.c - Momentics IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Run
bin.task_sync_condvars
qnxrtc
Project Explorer
child_process
hello_world
multi_thread
parent_process
pthread_counter
pthread_counter_fifo
pthread_counter_rr
qnxrtc
task_sync_condvars
Binaries
task_sync_condvars - [x86_64/le]
Includes
src
build
Makefile
task_sync_mutex
tsk_sync_semaphores
*task_sync_mutex.c
*tsk_sync_semaphores.c
task_sync_condvars.c
pthread_mutex_lock(&mtx);
system_ready = 1;
pthread_cond_broadcast(&cv); // Wake up all 4 waiting threads
pthread_mutex_unlock(&mtx);
for (int i = 0; i < 4; i++)
{
pthread_join(threads[i], NULL);
}
printf("Final Global Count: %d\n", global_count);
return 0;
}
Console
<terminated> bin.task_sync_condvars [C/C++ QNX Application] /tmp/task_sync_condvars on qnxrtc pid 2056194 (5/16/26, 12:29 AM)
Thread 1: Waiting for system_ready signal...
Thread 2: Waiting for system_ready signal...
Thread 3: Waiting for system_ready signal...
Thread 4: Waiting for system_ready signal...
Main: System is now READY. Signaling all threads...
Thread 1: Signal received! Performing task...
Thread 1: Incrementing global count to: 1
Thread 2: Signal received! Performing task...
Thread 2: Incrementing global count to: 2
Thread 3: Signal received! Performing task...
Thread 3: Incrementing global count to: 3
Thread 4: Signal received! Performing task...
Thread 4: Incrementing global count to: 4
Final Global Count: 4
Writable
Smart Insert
73 : 2 : 1822
137M of 266M

Procedure:

Step 1: Create a new QNX Executable project in the Momentics IDE.

Step 2: Use sched_setscheduler() to set the process to SCHED_RR with a priority of 10 to ensure all threads share the CPU time slices equally.

Step 3: Define a global mutex (mtx) and a condition variable (cv).

Step 4: Create 4 worker threads. Inside the thread function, use pthread_cond_wait() to put threads in a blocked state until a signal is received.

Step 5: Use sleep(2) in the main thread to simulate a system startup delay.

Step 6: Change the system_ready flag to 1 and use pthread_cond_broadcast() to wake all 4 threads simultaneously.

Step 7: Build the project and run it as a C/C++ QNX Application.

Step 8: Observe each thread waking up, acquiring the mutex, incrementing global_count, and printing the result to the console.

Results:

The experiment successfully demonstrated thread signaling using condition variables. The output verified that all 4 threads remained suspended until the broadcast signal was issued. Upon signaling, the threads executed the increment task sequentially under mutex protection, ensuring the global_count reached exactly 4 without race conditions.

Source Code (Server):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <process.h>

typedef struct
{
    int temperature;     // Unit: °C
    int humidity;        // Unit: %
    int soil_moisture;   // Unit: cb
} sensor_data_t;

typedef struct
{
    char acknowledgment[50];
} server_resp_t;

int main()
{
    int chid, rcvid;
    sensor_data_t msg;
    server_resp_t resp;

    chid = ChannelCreate(0);
    printf("Server Active. PID: %d, CHID: %d\n", getpid(), chid);

    while (1)
    {
        rcvid = MsgReceive(chid, &msg, sizeof(msg), NULL);

        printf("\n[Server] Received Update:\n");
        printf("Temp: %d°C | Hum: %d%% | Soil: %d cb\n",
                msg.temperature, msg.humidity, msg.soil_moisture);

        sprintf(resp.acknowledgment, "Server ACK: Values logged
successfully.");

        MsgReply(rcvid, 0, &resp, sizeof(resp));
    }
    return 0;
}
```

Expt. No.: 8A ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (NO NAME)
Date: USING MESSGAE PASSING

Aim:

To implement synchronous inter-process communication (IPC) between an independent Client and Server process in QNX Neutrino RTOS using manual PID/CHID tracking, transferring structured sensor data periodically.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm (Server):

Step 1: Start - The server application starts execution at the main() function.

Step 2: Chanel Creation - The server invokes ChannelCreate(0) to initialize a new communication channel on the QNX kernel.

Step 3: Expose Connection Info - The server retrieves its own Process ID using getpid() and prints both the PID and the CHID (Channel ID) to the console terminal so the client can find it.

Step 4: Enter Infinite Loop - The server enters a continuous while(1) loop to handle incoming data packets.

Step 5: Receive Message (Blocking) - The server executes MsgReceive(). The kernel puts the server into a RECEIVE-blocked state, consuming no CPU cycles until a message arrives on that channel.

Step 6: Process Data - When a message arrives, the server unblocks, reads the sensor_data_t structure, and prints the values formatted with their units (°C, %, cb) to the console.

Step 7: Formulate Reply - The server copies an acknowledgment string into the server_resp_t response structure.

Step 8: Unblock Client - The server calls MsgReply() using the received ID (rcvid), sending the response back and unblocking the client process.

Step 9: Repeat - The control loops back to Step 5 to await the next transmission.

Source Code (Client):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <unistd.h>

typedef struct
{
    int temperature;     // Unit: °C
    int humidity;        // Unit: %
    int soil_moisture;   // Unit: cb
} sensor_data_t;

typedef struct
{
  char acknowledgment[50];
} server_resp_t;

int main()
{
    int coid, server_pid, server_chid;
    sensor_data_t s_data = {25, 60, 30};
    server_resp_t resp;

    printf("Enter Server PID and CHID: ");
    scanf("%d %d", &server_pid, &server_chid);

    coid = ConnectAttach(0, server_pid, server_chid, 0, 0);

    printf("Client: Starting repeated transmission every 2 seconds...\n");

    while (1)
    {
        if (MsgSend(coid, &s_data, sizeof(s_data), &resp, sizeof(resp)) == -1)
        {
            break;
        }

        printf("Client: Server Response -> %s\n", resp.acknowledgment);

        sleep(2);
    }

    ConnectDetach(coid);
    return 0;
}
```

Algorithm (client):

Step 1: Start - The client application starts execution at the main() function and initializes its default sensor structure variables.

Step 2: Input Target Parameters - The program prompts the user to manually input the Server's PID and CHID printed on the server console.

Step 3:Establish Connection - The client passes these values into ConnectAttach(), creating a local connection ID (coid) acting as a direct path to the server's channel.

Step 4: Enter Infinite Loop - The client enters an infinite while(1) loop for periodic data transmission.

Step 5: Synchronous Data Send - The client calls MsgSend(), transmitting the sensor structure over the connection. The client process immediately enters a SEND-blocked state.

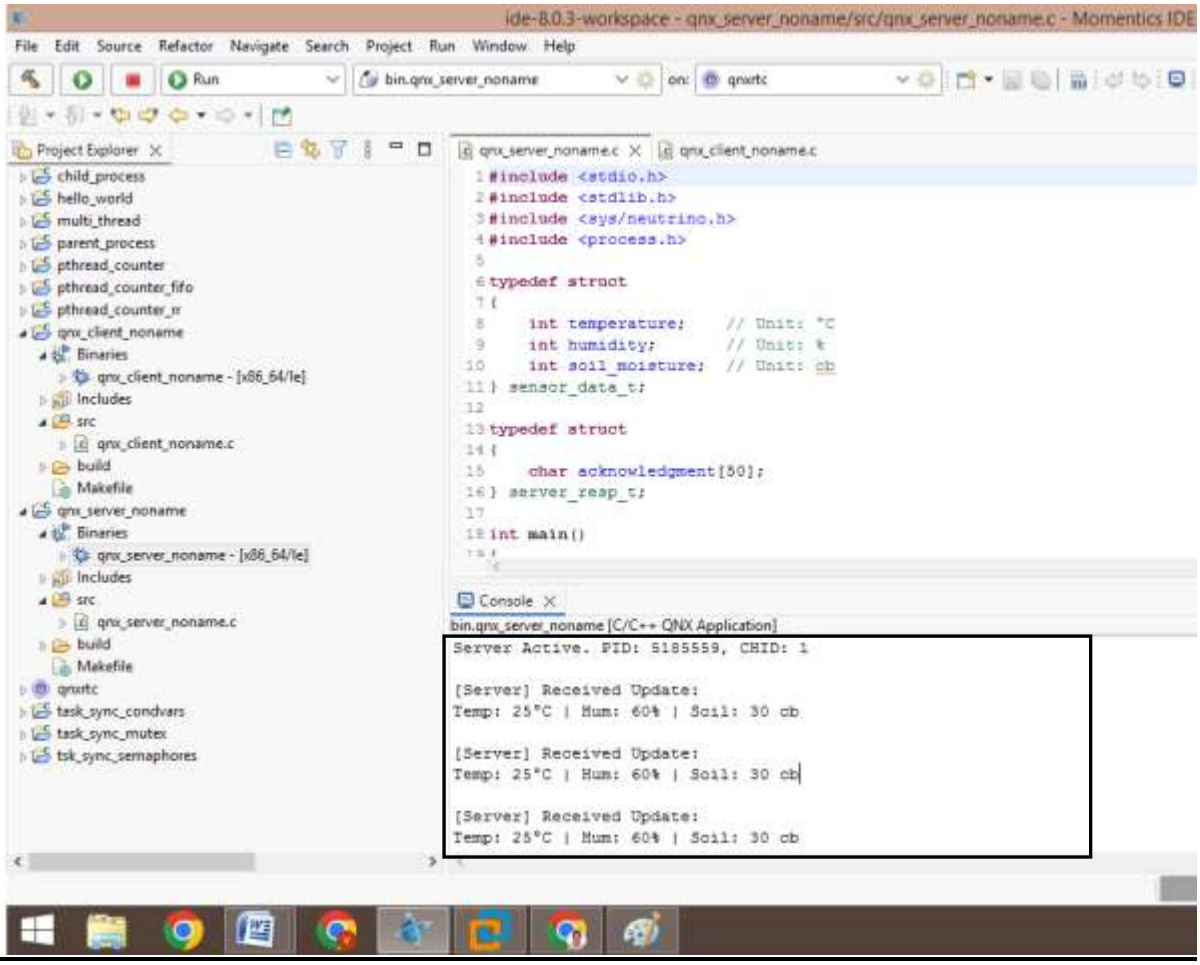
Step 6: Wait for Acknowledgment - The client stays suspended until the server processes the data and sends a reply.

Step 7: Process Response - Upon unblocking, the client reads the server's acknowledgment data from its response buffer and prints it to the terminal.

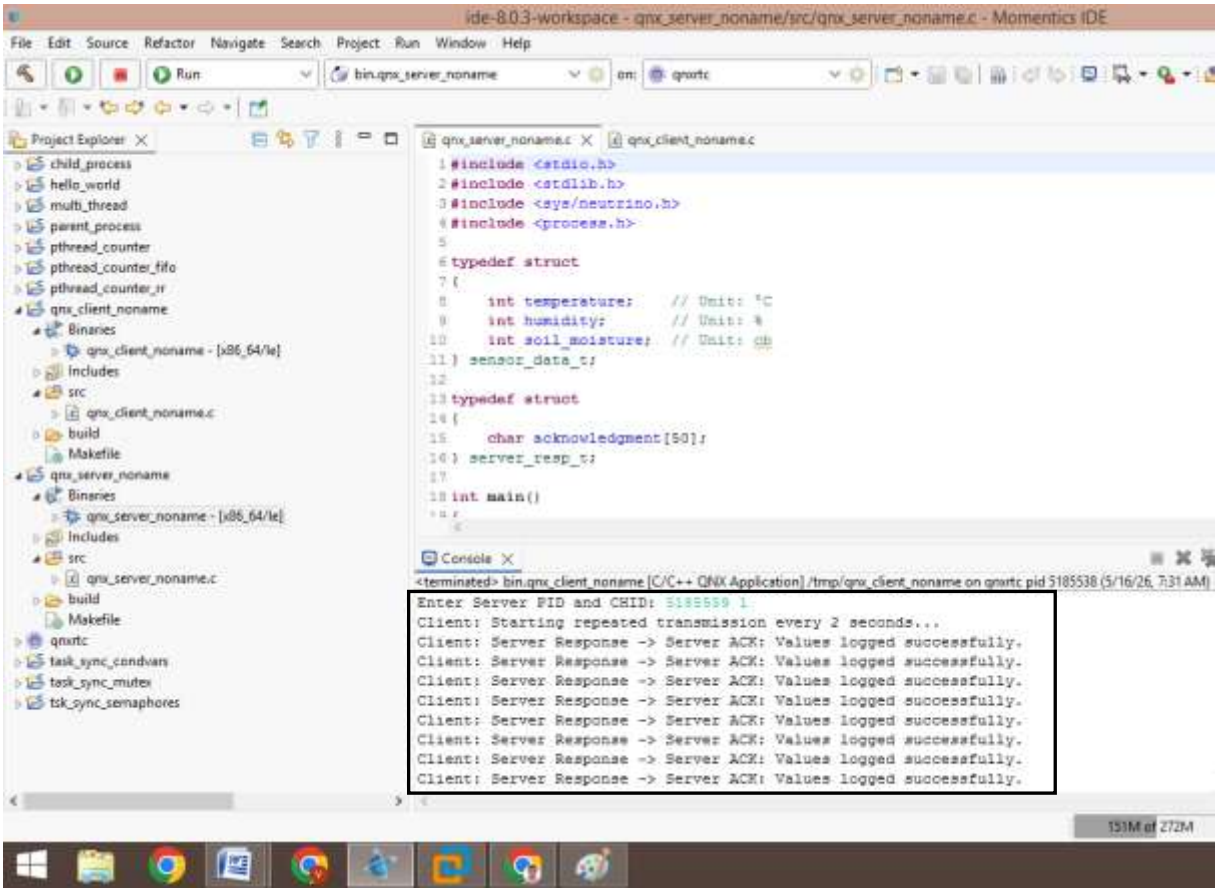
Step 8: Periodic Delay - The client calls sleep(2), placing itself into a timed-delay state for 2 seconds.

Step 9: Repeat - After 2 seconds, the control loops back to Step 5 to send the next batch of data.

Output (Server):



Output (Client):



Procedure:

Step 1: Build two separate QNX executable applications within the workspace: a server process and a client process.

Step 2: Launch the Server process first on the target architecture terminal. It creates a message channel and displays its system Process ID (PID) and Channel ID (CHID).

Step 3: Run the Client process. When prompted, manually pass the printed PID and CHID numbers via standard input.

Step 4: The Client invokes ConnectAttach() to create a local connection ID (coid) mapped to the server channel.

Step 5: The Client enters a loop sending sensor elements (Temperature, Humidity, Soil Moisture) every 2 seconds via MsgSend().

Step 6: Observe the server extracting values, formatting them with their respective units (°C, %, cb), and unblocking the client with an acknowledgment block.

Results:

Thus synchronous communication via standard messaging functions was established successfully. The Client accurately dispatched environment states at specific intervals, and the Server extracted and parsed the integers with correct dimensional units without dropouts or process collisions.

Source Code (Server):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/iofunc.h>
#include <sys/dispatch.h>

typedef struct
{
    int temperature, humidity, soil_moisture;
} sensor_data_t;

int main()
{
    name_attach_t *attach;
    sensor_data_t msg;
    int rcvid;

    if ((attach = name_attach(NULL, "Sensor_Server", 0)) == NULL)
    {
        return EXIT_FAILURE;
    }

    printf("Server 'Sensor_Server' is registered and listening...\n");

    while (1)
    {
        rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);

        if (rcvid == -1) break;

        printf("\n[Server] Automatic Lookup Success. Data Received:\n");
        printf("Temp: %d°C | Hum: %d%% | Soil: %d cb\n",
                msg.temperature, msg.humidity, msg.soil_moisture);

        MsgReply(rcvid, 0, "ACK", 4);
    }

    name_detach(attach, 0);
    return 0;
}
```

Expt. No.: 8B ESTABLISHING CONNECTION BETWEEN CLIENT AND SERVER (NAMED)
Date: USING MESSGAE PASSING

Aim:

To implement synchronous inter-process communication (IPC) between an independent Client and Server process in QNX Neutrino RTOS with automated server discovery in QNX Neutrino RTOS using the name_attach() and name_open() APIs, enabling a client to send sensor parameters to a server identified by a unique name string periodically.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm (Server):

Step 1: Start - The server application starts execution at the main() function.

Step 2: Register Global Identifier: The server calls name_attach(NULL, "Sensor_Server", 0). This registers the text name "Sensor_Server" inside the QNX namespace path-space (/dev/name/local/).

Step 3: Verification Check: If the registration returns NULL (name already taken or service failure), the program logs an error and terminates.

Step 4: Enter Receive Loop: The server drops into a continuous execution loop.

Step 5: Await Messages: The server calls MsgReceive() using the channel ID assigned automatically by the attachment structure (attach->chid). The process blocks.

Step 6: Message Validation: Once unblocked by an incoming message, the server checks if the returned receive ID (rcvid) is a system pulse or standard data (rcvid > 0).

Step 7: Read and Display: If it is valid client data, the server extracts the structure and prints the Temperature, Humidity, and Soil Moisture values alongside their technical units.

Step 8: Synchronous Reply: The server writes a confirmation message and delivers it via MsgReply(), releasing the client.

Step 9: Loop Reset: The control returns to Step 5. If the loop breaks, name_detach() is called to clean up the namespace before exiting.

Source Code (Client):

```
#include <stdio.h>
#include <sys/dispatch.h>
#include <unistd.h>

typedef struct
{
    int temperature, humidity, soil_moisture;
} sensor_data_t;

int main()
{
    int coid;
    sensor_data_t s_data = {24, 55, 40};
    char ack[10];

    printf("Client: Looking for 'Sensor_Server'...\n");

    while ((coid = name_open("Sensor_Server", 0)) == -1)
    {
        sleep(1); // Wait if server isn't started yet
    }

    printf("Client: Connected to Server. Starting 2s loop...\n");

    while (1)
    {
        MsgSend(coid, &s_data, sizeof(s_data), ack, sizeof(ack));
        printf("Client: Data Sent. Server says: %s\n", ack);

        s_data.temperature++; // Simulate variations
        sleep(2);
    }

    name_close(coid);
    return 0;
}
```

Algorithm (client):

Step 1: Start: The client application launches and configures its local sensor information structure.

Step 2: Automated Namespace Lookup: The client calls name_open("Sensor_Server", 0) to query the operating system for the service location..

Step 3: Connection Loop Check: If name_open() returns -1, it means the server has not initialized yet. The client enters a loop, running sleep(1) and retrying until the service name is resolved.

Step 4: Route Found: Once the QNX manager maps the string name to the hidden target channel keys, a local connection handler (coid) is automatically established.

Step 5: Enter Transmission Loop: The client enters a continuous loop layer.

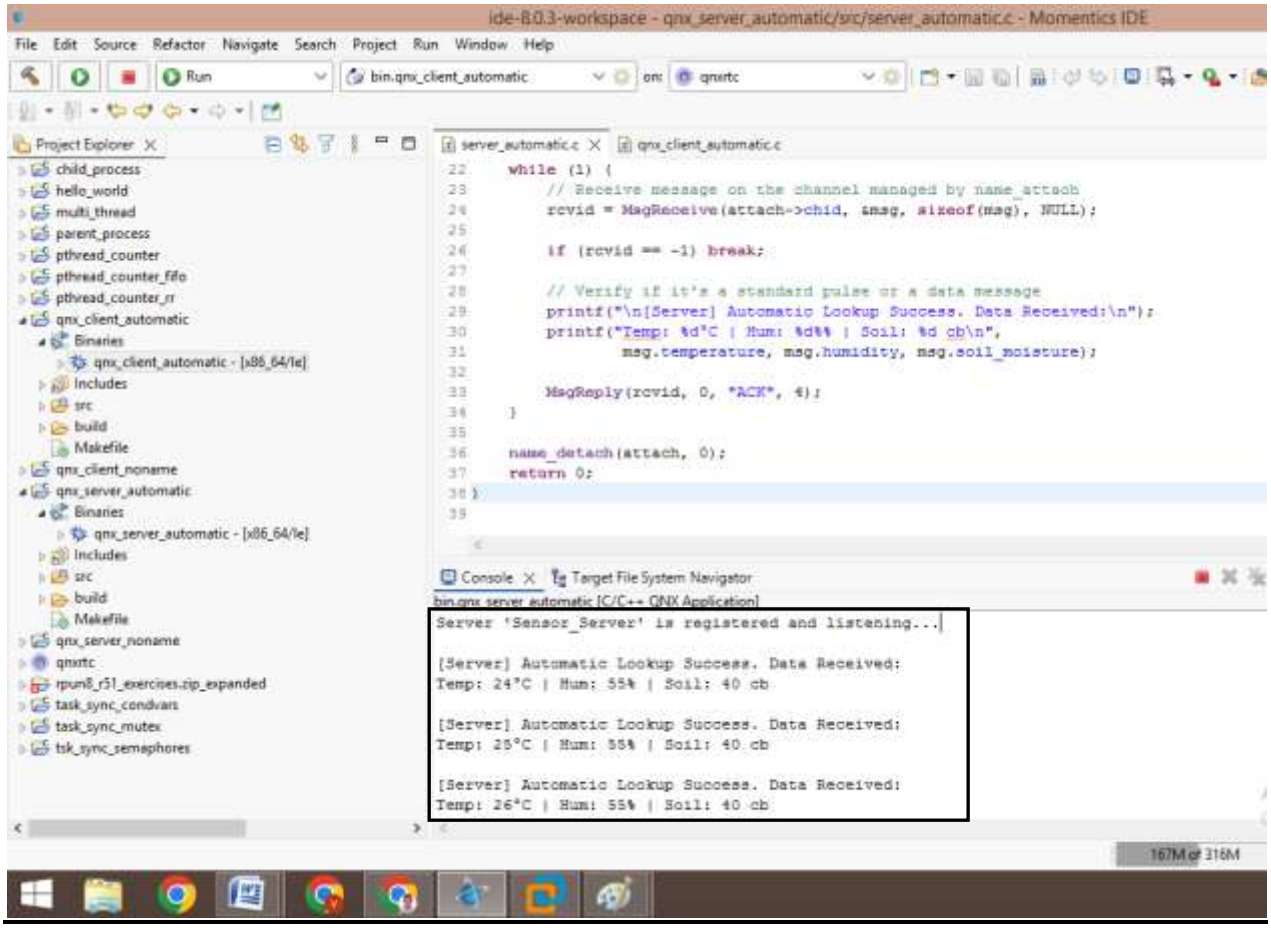
Step 6: Transmit Payload: The client passes the connection descriptor into MsgSend(). The client blocks instantly, handing off context management to the kernel.

Step 7: Receive Acknowledgment: The client wakes up only when the server returns a reply packet, dumping the acknowledgment text payload straight to the console window.

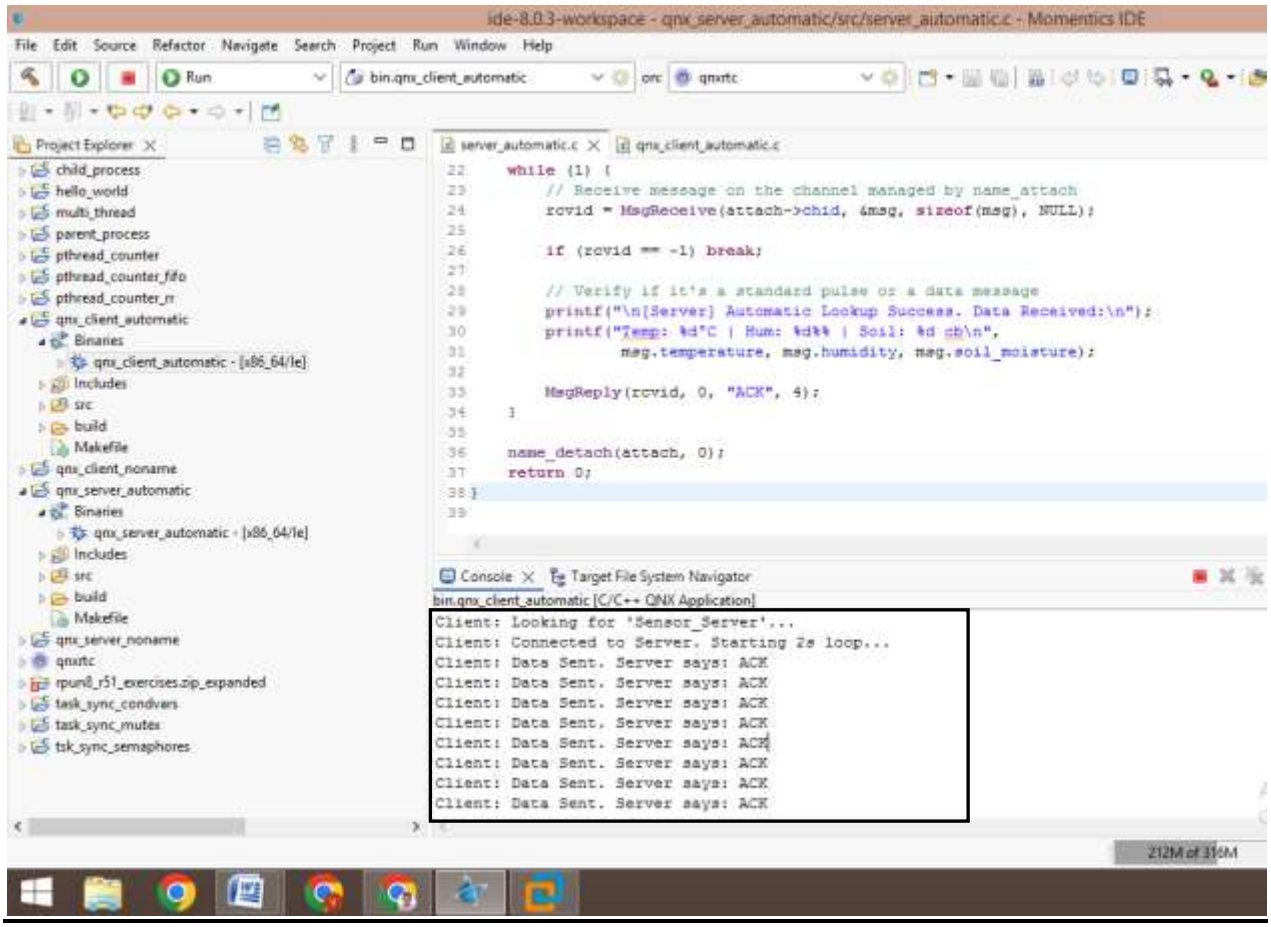
Step 8: Periodic Delay - The client calls sleep(2), placing itself into a timed-delay state for 2 seconds.

Step 9: Repeat - After 2 seconds, the control loops back to Step 6 to send the next batch of data.

Output (Server):



Output (Client):



Procedure:

Step 1: Build two separate QNX executable applications within the workspace: a server process and a client process.

Step 2: Launch the Server process first on the target architecture terminal. It creates a message channel and displays its system Process ID (PID) and Channel ID (CHID).

Step 3: Run the Client process. When prompted, manually pass the printed PID and CHID numbers via standard input.

Step 4: The Client invokes ConnectAttach() to create a local connection ID (coid) mapped to the server channel.

Step 5: The Client enters a loop sending sensor elements (Temperature, Humidity, Soil Moisture) every 2 seconds via MsgSend().

Step 6: Observe the server extracting values, formatting them with their respective units (°C, %, cb), and unblocking the client with an acknowledgment block.

Results:

Thus, the implementation of automated server discovery using the QNX Name Service was successfully completed, demonstrating that a client process can dynamically resolve a server's location via the global identifier "Sensor_Server" in the QNX path-space without any manual entry of Process IDs (PID) or Channel IDs (CHID). The Client accurately dispatched environment states at specific intervals, and the Server extracted and parsed the integers with correct dimensional units without dropouts or process collisions.

Source Code (Pulse Server):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <process.h>
#include <sched.h>

int main()
{
    int chid, rcvid;
    struct _pulse pulse_msg;
    struct sched_param param;

    param.sched_priority = 10;
    if (sched_setscheduler(0, SCHED_RR, &param) == -1)
    {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }

    chid = ChannelCreate(0);
    printf("Pulse Server Active. PID: %d, CHID: %d\n", getpid(), chid);
    printf("Awaiting asynchronous pulses...\n");

    while (1)
    {
        rcvid = MsgReceive(chid, &pulse_msg, sizeof(pulse_msg), NULL);

        if (rcvid == -1) {
            perror("MsgReceive failed");
            break;
        }

        if (rcvid == 0)
        {
            printf("\n[Receiver] Asynchronous Pulse Event Captured!\n");
            printf("Pulse Type/Code: %d\n", pulse_msg.code);
            printf("Pulse Associated Value: %d\n", pulse_msg.value.sival_int);
        }

        else
        {
            printf("[Receiver] Received standard message from rcvid: %d\n", rcvid);
            MsgReply(rcvid, 0, NULL, 0);
        }
    }
    return 0;
}
```

Expt. No.: 9 ASYNCHRONOUS NOTIFICATION HANDLING USING QNX PULSES

Date:

Aim:

To implement asynchronous notification handling in QNX Neutrino RTOS using QNX Pulses, enabling a non-blocking sender to transmit short, fixed-size notification events to a receiver process under Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm (Pulse Server):

Step 1: Start - The server application starts execution at the main() function.

Step 2: Scheduler Setup - Sets system thread scheduling context rules to Round Robin (SCHED_RR).

Step 3: Channel Mount - Instantiates an isolated communication endpoint using ChannelCreate(0).

Step 4: Console Display - Dumps runtime resolution parameters (PID and CHID) to stdout.

Step 5: Passive Block state - Invokes MsgReceive(). The thread is placed into a kernel wait state without consuming CPU execution intervals.

Step 6: Pulse Trapping - Upon receiving a notification payload, it evaluates if rcvid == 0.

Step 7: Extract Payload - If rcvid == 0, it unmarshals the internal _pulse structure array fields (code and value.sival_int).

Step 8: Render Output - Dumps parsed notification details onto the console and immediately restarts the polling loop.

Source Code (Pulse Client):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <unistd.h>
#include <sched.h>

int main()
{
    int coid, server_pid, server_chid;
    int event_counter = 100;
    int pulse_code = 55;
    struct sched_param param;

    param.sched_priority = 10;
    sched_setscheduler(0, SCHED_RR, &param);

    printf("Enter Target Server PID and CHID: ");
    if (scanf("%d %d", &server_pid, &server_chid) != 2)
    {
        return EXIT_FAILURE;
    }

    coid = ConnectAttach(0, server_pid, server_chid, 0, 0);
    if (coid == -1)
    {
        perror("ConnectAttach failed");
        return EXIT_FAILURE;
    }

    printf("Sender: Commencing asynchronous notifications every 2 seconds...\n");

    while (1)
    {
        printf("\nSender: Firing non-blocking pulse notification...\n");

        if (MsgSendPulse(coid, 10, pulse_code, event_counter) == -1)
        {
            perror("MsgSendPulse failed");
            break;
        }

        printf("Sender: Pulse issued successfully. Continuing execution
               instantly.\n");

        event_counter += 5;
        sleep(2);
    }

    ConnectDetach(coid);
    return 0;
}
```

Algorithm (Pulse Client):

Step 1: Start - The client program launches and sets up baseline metrics.

Step 2: Policy Configuration - Allocates priority context variables and implements SCHED_RR parameters.

Step 3: Manual Routing Input - Collects the target location data coordinates (Server PID & CHID) via terminal inputs.

Step 4: Node Binding - Passes coordinates to ConnectAttach() to establish a valid local connection descriptor link (coid).

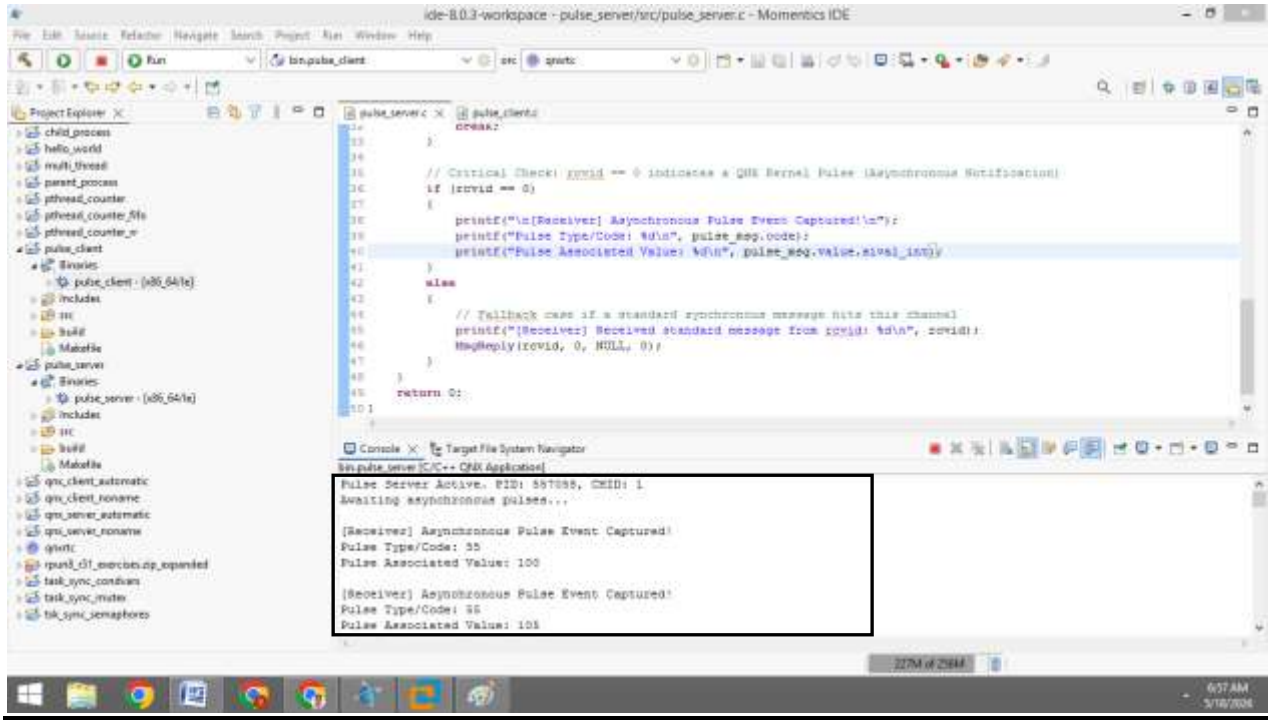
Step 5: Periodic Loop Phase - Drops the instruction pointer into a processing while(1) ring layer.

Step 6: Non-Blocking Fire Action - Calls the MsgSendPulse() function, pushing a small data frame containing priority information, an identity signature code, and an integer payload directly into the channel.

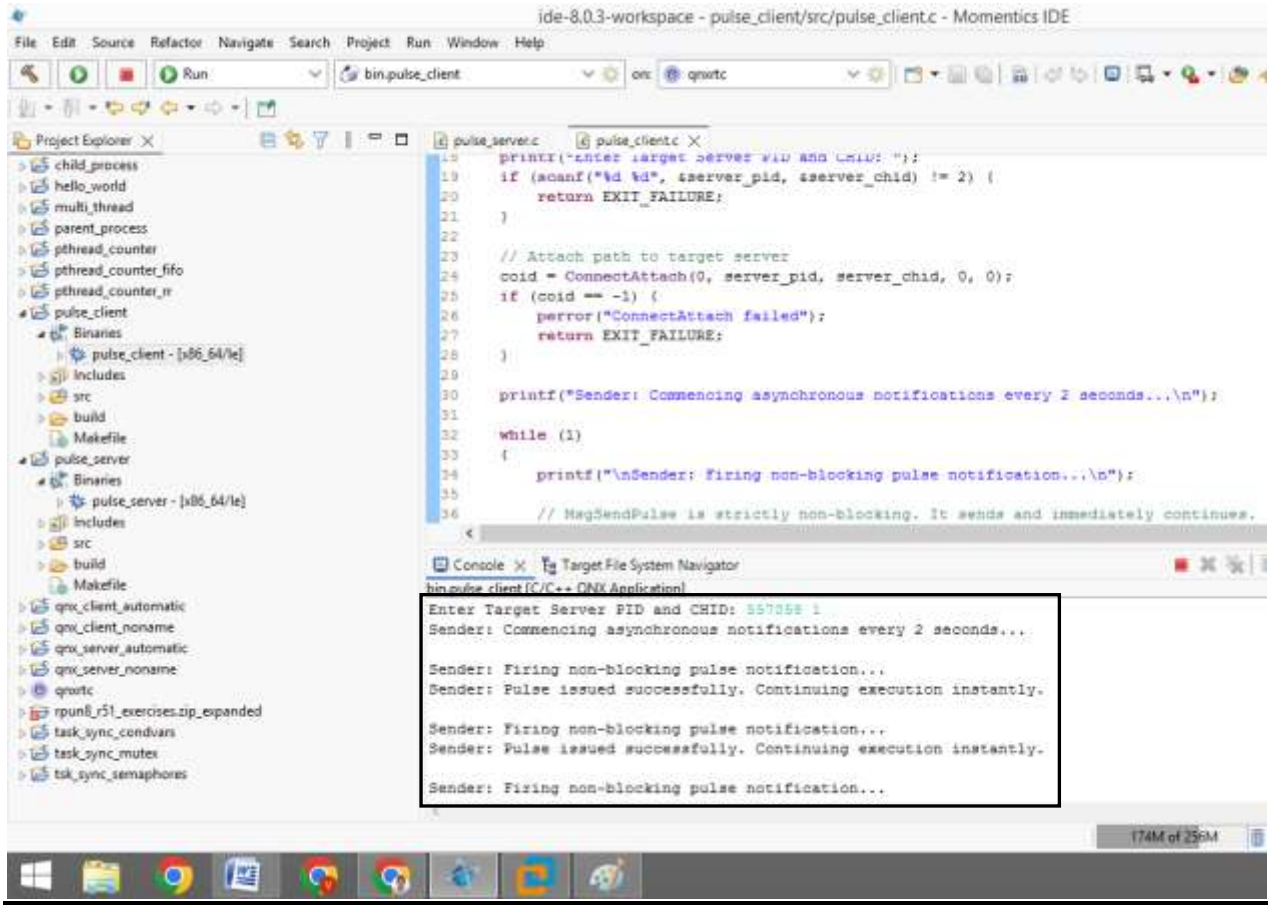
Step 7: Immediate Continuation - Because pulses are entirely asynchronous, the caller does not wait for a receiver response. It continues instantly to print status strings to stdout.

Step 8: Delay Interval - Executes a sleep(2) timeout to yield execution before iterating the sequence again.

Output (Pulse Server):



Output (Pulse Client):



Procedure:

Step 1: Create two separate QNX Executable projects in Momentics IDE: Pulse_Server (Receiver) and Pulse_Client (Sender).

Step 2: Configure Scheduler: In both programs, use sched_setscheduler() to enforce SCHED_RR with a priority of 10 to guarantee uniform real-time time-slicing.

Step 3: Setup Server as follows:

- Create a communication channel using ChannelCreate().
- Drop into an execution loop using MsgReceive().
- When a pulse is captured, evaluate the receipt index (rcvid == 0) and decode the pulse structure containing the unique code and 8-bit or 32-bit data payload.

Step 4: Setup client as follows:

- Establish a connection handle to the receiver's process space via ConnectAttach().
- Replace the blocking MsgSend() API with MsgSendPulse().
- Configure the transmission parameters to pass a custom pulse code (e.g., 55) and an integer value representing an asynchronous event status.

Step 5: Run the Pulse_Server first to yield its PID and CHID, then trigger the Pulse_Client to fire non-blocking notification bursts every 2 seconds using sleep(2).

Step 6: Observe the server extracting values, formatting them with their respective units (°C, %, cb), and unblocking the client with an acknowledgment block.

Results:

Thus, the implementation of asynchronous notification handling using QNX Pulses under Round Robin scheduling was successfully completed. The client process demonstrated non-blocking behavior by continuously transmitting data events every 2 seconds without ever shifting into a send-blocked state. The server correctly trapped each independent notification event through the message layer, successfully verifying pulse boundaries (rcvid == 0) to extract and display codes and values on the terminal window.

Source Code (Shared Memory Writer):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <sched.h>
#define SHM_PATH "/shm_sensor_data"

typedef struct
{
    int temperature;    // Unit: °C
    int humidity;       // Unit: %
    int soil_moisture;  // Unit: cb
    int update_flag;    // Sequence counter to track changes
} shared_data_t;

int main()
{
    int shm_fd;    shared_data_t *ptr;    struct sched_param param;
    param.sched_priority = 10;
    if (sched_setscheduler(0, SCHED_RR, &param) == -1) {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }
    shm_fd = shm_open(SHM_PATH, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open failed");
        exit(EXIT_FAILURE);
    }
    if (ftruncate(shm_fd, sizeof(shared_data_t)) == -1)
    {
        perror("ftruncate failed");
        exit(EXIT_FAILURE);
    }
    ptr = (shared_data_t *)mmap(NULL, sizeof(shared_data_t),
                                PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED)
    {
        perror("mmap failed");
        exit(EXIT_FAILURE);
    }

    printf("Shared Memory Writer Active. Path: /dev/shmem%s\n", SHM_PATH);

    ptr->temperature = 26;    ptr->humidity = 58;    ptr->soil_moisture = 34;
    ptr->update_flag = 0;

    while (1)
    {
        ptr->temperature++;   ptr->humidity--;  ptr->update_flag++;

        printf("\n[Writer] Shared Memory Segment Updated (Seq: %d)\n", ptr-
>update_flag);
        printf("Writing -> Temp: %d°C | Hum: %d%% | Soil: %d cb\n",
                ptr->temperature, ptr->humidity, ptr->soil moisture);

        sleep(2);
    }
    munmap(ptr, sizeof(shared_data_t));
    close(shm_fd);
    return 0;
}
```

Expt. No.: 10 **HIGH-SPEED DATA EXCHANGE USING SHARED MEMORY**
Date: **AND SHARED OBJECTS**

Aim:

To implement high-speed data exchange between two independent processes (Writer and Reader) in QNX Neutrino RTOS using POSIX Shared Memory Objects, enabling rapid data transfer without the overhead of kernel message copying under Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm (Shared Memoory Writer):

Step 1: Start - The program starts execution at the main() entry block.

Step 2: Policy Configuration - Configures system priority attributes and enforces Round Robin (SCHED_RR) scheduling.

Step 3: Object Initialization - Invokes shm_open() to allocate a shared memory file descriptor node under /dev/shmem/shm_sensor_data.

Step 4: Dimension Sizing - Executes ftruncate() to establish memory allocation sizing parameters equivalent to the structure blueprint size.

Step 5: Memory Mapping - Calls mmap() to register the shared allocation directly inside its local virtual memory address table.

Step 6: Data Loop - Enters a continuous execution loop ring to update environmental data periodically.

Step 7: Write and Modify - Directly writes the updated integers into the mapped memory structure pointer and bumps the update sequence token flag.

Step 8: Delay State - Pauses via sleep(2) to hand over CPU priority slots under the Round Robin scheduler before repeating the write process.

Source Code (Shared Memory Reader):

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <sched.h>
#define SHM_PATH "/shm_sensor_data"

typedef struct
{
    int temperature;    // Unit: °C
    int humidity;       // Unit: %
    int soil_moisture;  // Unit: cb
    int update_flag;    // Sequence counter to track changes
} shared_data_t;

int main()
{
    int shm_fd;
    shared_data_t *ptr;
    struct sched_param param;
    param.sched_priority = 10;
    sched_setscheduler(0, SCHED_RR, &param);
    while ((shm_fd = shm_open(SHM_PATH, O_RDONLY, 0666)) == -1)
    {
        printf("Waiting for Shared Memory Object segment creation...\n");
        sleep(1);
    }

    ptr = (shared_data_t *)mmap(NULL, sizeof(shared_data_t),
                                PROT_READ, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED)
    {
        perror("mmap failed");
        exit(EXIT_FAILURE);
    }

    printf("Shared Memory Reader Active. Connected successfully.\n");

    int last_seen_seq = -1;
    while (1)
    {
        if (ptr->update_flag != last_seen_seq)
        {
            printf("\n[Reader] High-Speed Shared Memory Read Success:\n");
            printf("Temp: %d°C | Hum: %d%% | Soil: %d cb (Seq: %d)\n",
                    ptr->temperature, ptr->humidity, ptr->soil_moisture, ptr-
>update_flag);
            last_seen_seq = ptr->update_flag;
        }
        else
        {
            printf("[Reader] No new updates detected in shared memory
slot.\n");
        }

        sleep(2);
    }
    munmap(ptr, sizeof(shared_data_t));
    close(shm_fd);
    shm_unlink(SHM_PATH); // Unlink the object from system namespace
    return 0;
}
```

Algorithm (client):

Step 1: Start: The reader process initiates execution parameters.

Step 2: Scheduler Setup: Sets process priority profiles to conform to Round Robin (SCHED_RR) scheduling constraints.

Step 3: Namespace Discovery: Attempts to call shm_open() to attach to the existing path node identifier. If it returns -1, it delays for 1 second and retries.

Step 4: Local Pointer Mapping: Invokes mmap() to link the global object segment directly to the local reader address space pointer.

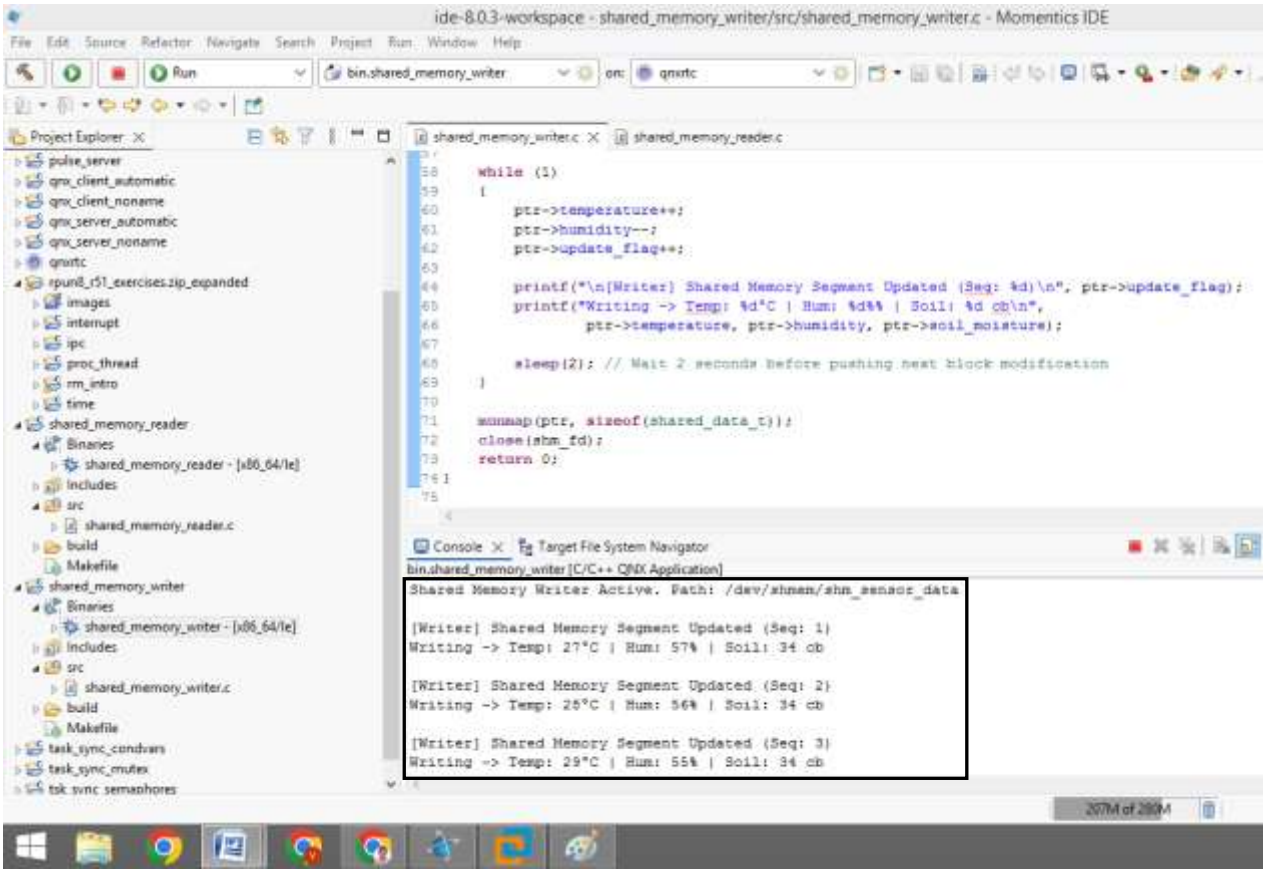
Step 5: Periodic Parsing Loop: Drops execution flow into an infinite observation loop.

Step 6: Update Condition Evaluation: Compares the structure's update_flag tracking variable against the local sequence history token register.

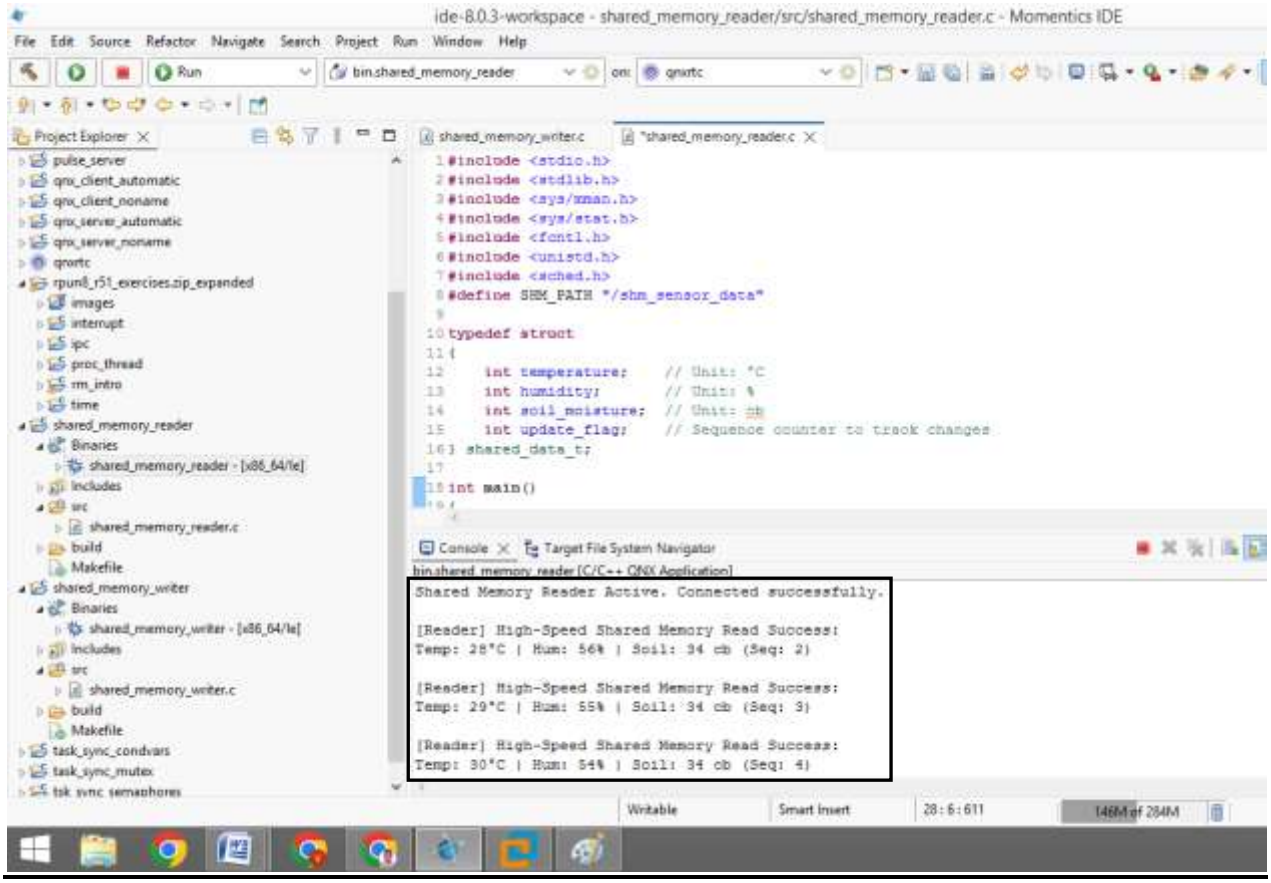
Step 7: Terminal Rendering: If the sequence flag changes, it reads the data block instantly without kernel message copy calls and displays the properties along with the monitoring units (°C, %, cb).

Step 8: Polled Timing Delay: Executes a sleep(2) command to space out read cycles before repeating.

Output (Shared Memory Writer):



Output (Shared Memory Reader):



Procedure:

Step 1: Initialize Projects: Create two separate QNX Executable projects in Momentics IDE: Shm_Writer and Shm_Reader.

Step 2: Configure Scheduler: In both programs, use sched_setscheduler() to enforce SCHED_RR with a priority of 10 to ensure fair, real-time time-slicing.

Step 3: Setup Shared Object Path: Establish a unique shared memory object name path (e.g., "/shm_sensor_data"). This path is created under the QNX shared memory namespace (/dev/shmem/).

Step 4: Setup Writer Process as follows:

- Create the shared memory object using shm_open() with the O_CREAT | O_RDWR flags.
- Define the allocation size of the memory block using ftruncate().
- Map the memory object into the process's address space using mmap().
- Write data parameters (Temperature, Humidity, Soil Moisture) into the mapped region every 2 seconds.

Step 5: Setup Reader Process as follows:

- Open the existing memory object using shm_open() with the O_RDONLY flag.
- Map the shared object into its own address space using mmap().
- Read, format, and display the sensor data parameters alongside their units every 2 seconds.

Step 6: Execution and Cleanup: Execute the Writer first to create and truncate the shared block, followed by the Reader. Upon termination, use shm_unlink() to clear the shared object from the OS namespace.

Results:

Thus, the implementation of high-speed data exchange using Shared Memory and Shared Objects under Round Robin scheduling was successfully completed. The Writer process effectively modified the target block metrics periodically, and the Reader process successfully mapped the memory file descriptor node to access the shared area without kernel intervention or structural duplication overhead. The console output verified that data was modified and read seamlessly across process boundaries every 2 seconds, displaying values accurately alongside their designated real-time tracking units.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <sys/syspage.h>
#include <hw/inout.h> // Required for in8() to read hardware ports
#include <unistd.h>
#include <sched.h>
#include <time.h>

#define HW_INTERRUPT_VECTOR 1  // Mapped to x86 Hardware Keyboard Controller
(IRQ 1)
#define KEYBOARD_DATA_PORT  0x60 // Standard x86 Keyboard Data Register Port

int global_event_count = 0;
int simulated_temperature = 25; // Unit: °C

int main() {
    int chid, rcvid;
    int interrupt_id;
    struct sigevent event;
    struct _pulse pulse_msg;
    struct sched_param param;

    uint64_t last_interrupt_time = 0;
    uint64_t current_time;

    param.sched_priority = 15;
    if (sched_setscheduler(0, SCHED_RR, &param) == -1)
    {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }

    if (ThreadCtl(_NTO_TCTL_IO_PRIV, 0) == -1)
    {
        perror("ThreadCtl I/O privileges failed. Run as root/superuser.");
        exit(EXIT_FAILURE);
    }

    chid = ChannelCreate(0);
    if (chid == -1)
    {
        perror("ChannelCreate failed");
        exit(EXIT_FAILURE);
    }

    int coid = ConnectAttach(0, 0, chid, _NTO_SIDE_CHANNEL, 0);
    SIGEV_PULSE_INIT(&event, coid, SIGEV_PULSE_PRIO_INHERIT,
_PULSE_CODE_MINAVAIL, 0);

    interrupt_id = InterruptAttachEvent(HW_INTERRUPT_VECTOR, &event, 0);
    if (interrupt_id == -1)
    {
        perror("InterruptAttachEvent failed");
        exit(EXIT_FAILURE);
    }

    printf("Interrupt Service Thread (IST) with Software Filtering
Active.\n");
    printf("Monitoring IRQ Vector %d (Keyboard) for clean key-press
events...\n", HW_INTERRUPT_VECTOR);
```

Expt. No.: 11 **INTERRUPT SERVICE ROUTINE (ISR) IMPLEMENTATION FOR**

Date: **EXTERNAL HARDWARE EVENTS**

Aim:

To implement an Interrupt Service Routine (ISR) in QNX Neutrino RTOS to handle keyboard hardware events via IRQ 1, incorporating software filtering (scancode tracking) and time-delta debouncing within an Interrupt Service Thread (IST) under Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)
- Standard AT Keyboard / Emulated Virtual Keyboard

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Step 1: Start - Program begins execution and requests I/O privileges (_NTO_TCTL_IO_PRIV) via ThreadCtl().

Step 2: Threading Optimization - Enforces the Round Robin policy (SCHED_RR) at priority level 15.

Step 3: Map Vector Event Path - Creates a communication channel, initializes a pulse block with code matching _PULSE_CODE_MINAVAIL, and links it to IRQ 1 via InterruptAttachEvent().

Step 4: Enter Passive Listening - Drops into a while(1) ring loop and suspends execution at MsgReceive().

Step 5: Intercept Trigger Event - When a keystroke occurs, the kernel maps the hardware line, masks further events, and forwards a notification pulse to unblock the thread execution.

Step 6: Register Read - The IST unblocks and instantly invokes in8(0x60) to pull the byte configuration from the keyboard data register. It also fetches the current system time in nanoseconds via ClockTime().

Step 7: Filter Break Conditions - Evaluates whether scancode & 0x80 is true. If it matches, the entry is flagged as a key release event; the thread calls InterruptUnmask() and skips to step 4 without modifying variables.

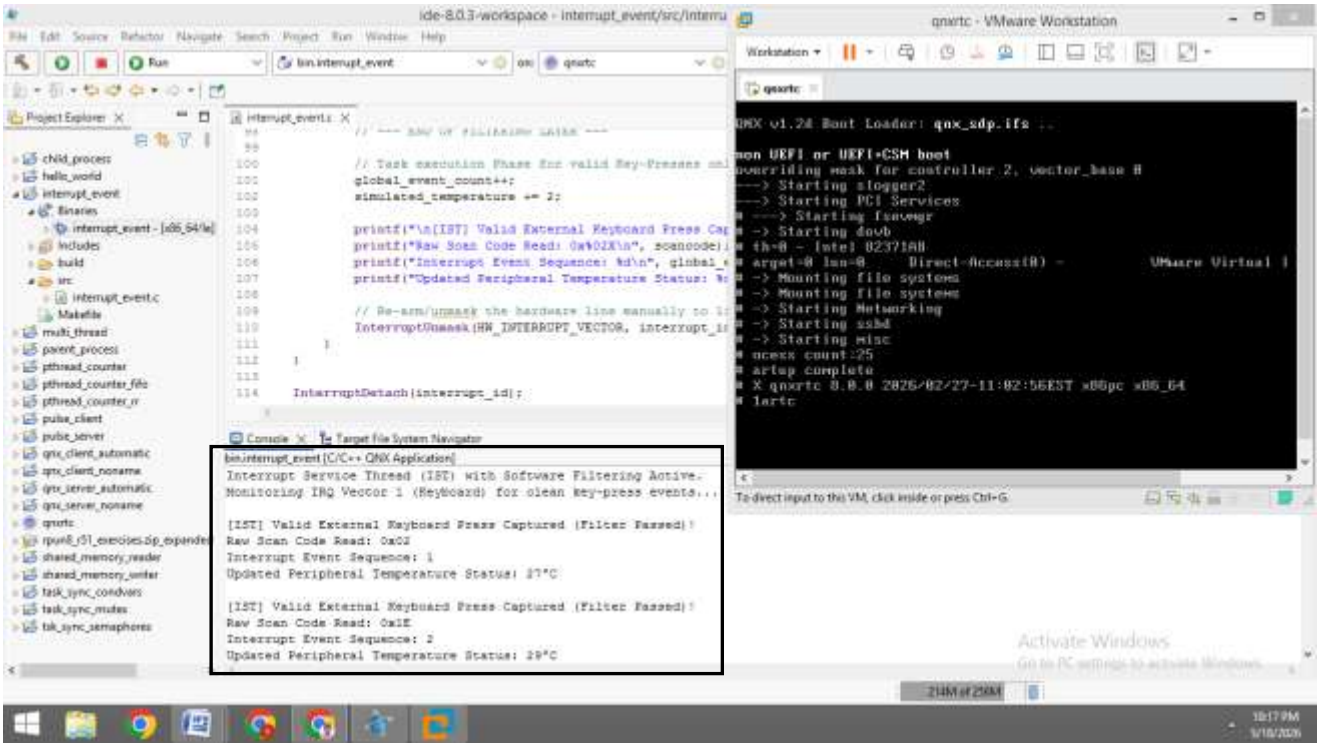
```
    while (1)
      {
          rcvid = MsgReceive(chid, &pulse_msg, sizeof(pulse_msg), NULL);

          if (rcvid == -1)
          {
              perror("MsgReceive failed");
              break;
          }

          if (rcvid == 0 && pulse_msg.code == _PULSE_CODE_MINAVAIL)
          {
              uint8_t scancode = in8(KEYBOARD_DATA_PORT);
              ClockTime(CLOCK_REALTIME, NULL, &current_time);

              if (scancode & 0x80)
              {
                  InterruptUnmask(HW_INTERRUPT_VECTOR, interrupt_id);
                  continue;
              }

              if ((current_time - last_interrupt_time) < 50000000)
              {
                  InterruptUnmask(HW_INTERRUPT_VECTOR, interrupt_id);
                  continue;
              }

              last_interrupt_time = current_time;

              global_event_count++;
              simulated_temperature += 2;

              printf("\n[IST] Valid External Keyboard Press Captured (Filter
                      Passed)!\n");
              printf("Raw Scan Code Read: 0x%02X\n", scancode);
              printf("Interrupt Event Sequence: %d\n", global_event_count);
              printf("Updated Peripheral Temperature Status: %d°C\n",
                       simulated_temperature);
              InterruptUnmask(HW_INTERRUPT_VECTOR, interrupt_id);
          }
      }
      InterruptDetach(interrupt_id);
      return 0;
  }
```

Step 7: Evaluate Time Threshold (Debounce) - Calculates the difference between current_time and last_interrupt_time. If the delta is less than 50,000,000 ns (50 ms), it is treated as mechanical switch noise; the thread drops it, unmasks the vector line, and skips to step 4.

Step 8: Update State Variable Context - If all filtering parameters pass, the current timestamp is saved, global_event_count is incremented, and simulated_temperature is bumped by 2.

Step 9: Display & Unmask - Outputs the valid raw scan code alongside the logged count and the updated metric unit (°C) to the terminal console. It then calls InterruptUnmask() to unblock the interrupt line before looping back to step 4.

Output:


ide-8.0.3-workspace - interrupt_event/src/interru
qnxrtc - VMware Workstation
bin.interrupt_event [C/C++ QNX Application]
Interrupt Service Thread (IST) with Software Filtering Active.
Monitoring IRQ Vector 1 (Keyboard) for clean key-press events...
[IST] Valid External Keyboard Press Captured (Filter Passed)!
Raw Scan Code Read: 0x02
Interrupt Event Sequence: 1
Updated Peripheral Temperature Status: 27°C
[IST] Valid External Keyboard Press Captured (Filter Passed)!
Raw Scan Code Read: 0x1E
Interrupt Event Sequence: 2
Updated Peripheral Temperature Status: 28°C

Procedure:

Step 1: Create a new "QNX Executable" project in Momentics IDE.

Step 2: Ensure the <hw/inout.h> system header is declared to gain access to the low-level in8() assembly-wrapper inline macro.

Step 3: Execute ThreadCtl(_NTO_TCTL_IO_PRIV, 0) at the start of main() to bypass standard ring-3 protections, enabling direct access to x86 I/O ports and interrupt tables.

Step 4: Enforce Scheduling Constraints: Configure a sched_param structure to enforce SCHED_RR (Round Robin) time-slicing at an elevated execution priority of 15.

Step 5: Bind a local communication channel and initialize an architecture-safe pulse configuration block using SIGEV_PULSE_INIT().

Step 6: Call InterruptAttachEvent() to register standard keyboard controller vector line 1.

Step 7: Inside the processing loop, use in8(0x60) to parse the raw keyboard matrix state. Apply bitwise screening (scancode & 0x80) to drop break codes, and calculate time deltas with ClockTime() to drop mechanical bounces.

Step 8: For all conditions (filtered loops or valid increments), call InterruptUnmask() to signal completion to the kernel-level interrupt controller.

Step 9: Build the binary file, move it to the target workspace filesystem, log in as root, and execute it while observing the console when typing on the keyboard.

Results:

Thus, the implementation of an Interrupt Service Routine (ISR) with integrated software filtering and time debouncing under Round Robin scheduling was successfully completed. By reading from I/O register port 0x60 and applying a bitwise mask alongside a nanosecond time guard, the program successfully eliminated redundant inputs caused by contact bounce and key release (Break) scan codes. The console terminal verified that the counter updated exactly **once** per discrete keystroke, demonstrating reliable and robust hardware event handling on the QNX Neutrino target platform.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/iofunc.h>
#include <sys/dispatch.h>
#include <unistd.h>
#include <sched.h>
#include <string.h>

#define BUFFER_SIZE 100
char device_buffer[BUFFER_SIZE] = "Initial Virtual Device Data\n";
int data_length = 28;

int io_read(resmgr_context_t *ctp, io_read_t *msg, iofunc_ocb_t *ocb)
{
    int status;

    if ((status = iofunc_read_verify(ctp, msg, ocb, NULL)) != EOK)
    {
        return status;
    }

    if (msg->i.xtype & _IO_XTYPE_MASK)
    {
        return ENOSYS;
    }

    if (ocb->offset >= data_length)
    {
        _IO_SET_READ_NBYTES(ctp, 0);
        return (_RESMGR_NPARTS(0));
    }

    int bytes_to_read = data_length - ocb->offset;
    if (bytes_to_read > msg->i.nbytes)
    {
        bytes_to_read = msg->i.nbytes;
    }

    printf("[Device Manager] Intercepted read(). Transferring %d bytes to
client...\n", bytes_to_read);

    SETIOV(ctp->iov, device_buffer + ocb->offset, bytes_to_read);
    _IO_SET_READ_NBYTES(ctp, bytes_to_read);

    ocb->offset += bytes_to_read;

    return (_RESMGR_NPARTS(1));
}
```

Expt. No.: 12 DESIGNING A RESOURCE MANAGER FOR QNX TARGET DEVICE

Date:

Aim:

To design and implement a basic POSIX-compliant Resource Manager in QNX Neutrino RTOS to manage access to a virtual or physical device by intercepting standard I/O system calls (like open, read, and write) under Round Robin (RR) scheduling.

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)
- Standard AT Keyboard / Emulated Virtual Keyboard

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm - POSIX Read Lifecycle Hook (io_read):

Step 1: Request Interception - When a client application passes /dev/virt_device into cat or read(), the kernel routes a read request message, waking up the manager thread.

Step 2: Verify Permissions - Validates read permissions using iofunc_read_verify().

Step 3: Boundary Offset Check - Compares the Client's open descriptor position (ocb->offset) against the available payload size (data_length). If the offset matches the data size, it signals End-of-File (EOF) by returning 0 bytes.

Step 4: Data Delivery - Copies data from the target memory window (device_buffer) to the client space using SETIOV() and _IO_SET_READ_NBYTES().

Step 5: Advance Offset - Adds the number of transmitted bytes to ocb->offset so subsequent read operations can track file position correctly, and exits back to the listener loop.

```
int io_write(resmgr_context_t *ctp, io_write_t *msg, iofunc_ocb_t *ocb)
{
    int status;

    if ((status = iofunc_write_verify(ctp, msg, ocb, NULL)) != EOK)
    {
        return status;
    }

    if (msg->i.xtype & _IO_XTYPE_MASK)
    {
        return ENOSYS;
    }

    int bytes_to_write = msg->i.nbytes;
    if (bytes_to_write > (BUFFER_SIZE - 1))
    {
        bytes_to_write = BUFFER_SIZE - 1;
    }

    printf("[Device Manager] Intercepted write(). Capturing input
stream...\n");

    resmgr_msgread(ctp, device_buffer, bytes_to_write, sizeof(msg->i));
    device_buffer[bytes_to_write] = '\0'; // Append string null terminator
    data length = bytes to write;         // Reset internal index size
tracking metric
    ocb->offset = 0;                       // Reset file position offset
tracker

    _IO_SET_WRITE_NBYTES(ctp, bytes_to_write);
    return (_RESMGR_NPARTS(0));
}
```

Algorithm - POSIX Write Lifecycle Hook (io_write):

Step 1: Request Interception: When a client app uses echo or write() on the device file, an incoming write message unblocks the manager thread.

Step 2: Verify Permissions: Validates write permissions using iofunc_write_verify().

Step 3: Extract Payload: Reads the incoming data stream directly out of the message container context buffer using resmgr_msgread().

Step 4: Buffer Mutate: Overwrites the contents of device_buffer, appends a string null terminator, and updates data_length with the new input byte count.

Step 5: Reset Offset Tracker: Sets ocb->offset = 0 so future read requests start from the beginning of the newly written string. It then acknowledges the operation via _IO_SET_WRITE_NBYTES().

```
int main()
{
    dispatch_t *dpp;
    resmgr_attr_t resmgr_attr;
    resmgr_context_t *ctp;
    resmgr_connect_funcs_t connect_funcs;
    resmgr_io_funcs_t io_funcs;
    iofunc_attr_t attr;
    struct sched_param param;

    param.sched_priority = 10;
    if (sched_setscheduler(0, SCHED_RR, &param) == -1)
    {
        perror("sched_setscheduler failed");
        exit(EXIT_FAILURE);
    }

    if (ThreadCtl(_NTO_TCTL_IO_PRIV, 0) == -1)
    {
        perror("ThreadCtl privileges failed. Run as root.");
        exit(EXIT_FAILURE);
    }

    if ((dpp = dispatch_create()) == NULL)
    {
        perror("dispatch_create failed");
        exit(EXIT_FAILURE);
    }

    memset(&resmgr_attr, 0, sizeof(resmgr_attr));
    resmgr_attr.nparts_max = 1;
    resmgr_attr.msg_max_size = 2048;

    iofunc_func_init(_RESMGR_CONNECT_NFUNCS, &connect_funcs,
_RESMGR_IO_NFUNCS, &io_funcs);

    io_funcs.read = io_read;
    io_funcs.write = io_write;

    iofunc_attr_init(&attr, S_IFCHR | 0666, NULL, NULL);
    attr.nbytes = data_length;

    if (resmgr_attach(dpp, &resmgr_attr, "/dev/virt_device", _FTYPE_ANY, 0,
                      &connect_funcs, &io_funcs, &attr) == -1)
    {
        perror("resmgr_attach failed");
        exit(EXIT_FAILURE);
    }

    if ((ctp = resmgr_context_alloc(dpp)) == NULL)
    {
        perror("resmgr_context_alloc failed");
        exit(EXIT_FAILURE);
    }

    printf("Resource Manager Active. Virtual device path mounted at:
/dev/virt_device\n");
    printf("Listening for POSIX open(), read(), and write() operations...\n");
```

Algorithm - POSIX Write Lifecycle Hook (io_write):

Step 1: Start - The driver initializes code boundaries at the main() functional block entry point.

Step 2: Enforce Scheduling Policy - Configures process context structures to enforce Round Robin (SCHED_RR) scheduling at execution priority 10.

Step 3: Elevate OS Access Level - Invokes ThreadCtl() to gain the root-level I/O privilege authorizations necessary to instantiate new device files inside the /dev/ directory tree.

Step 4: Dispatch Structure Creation - Calls dispatch_create() to construct a message polling mechanism.

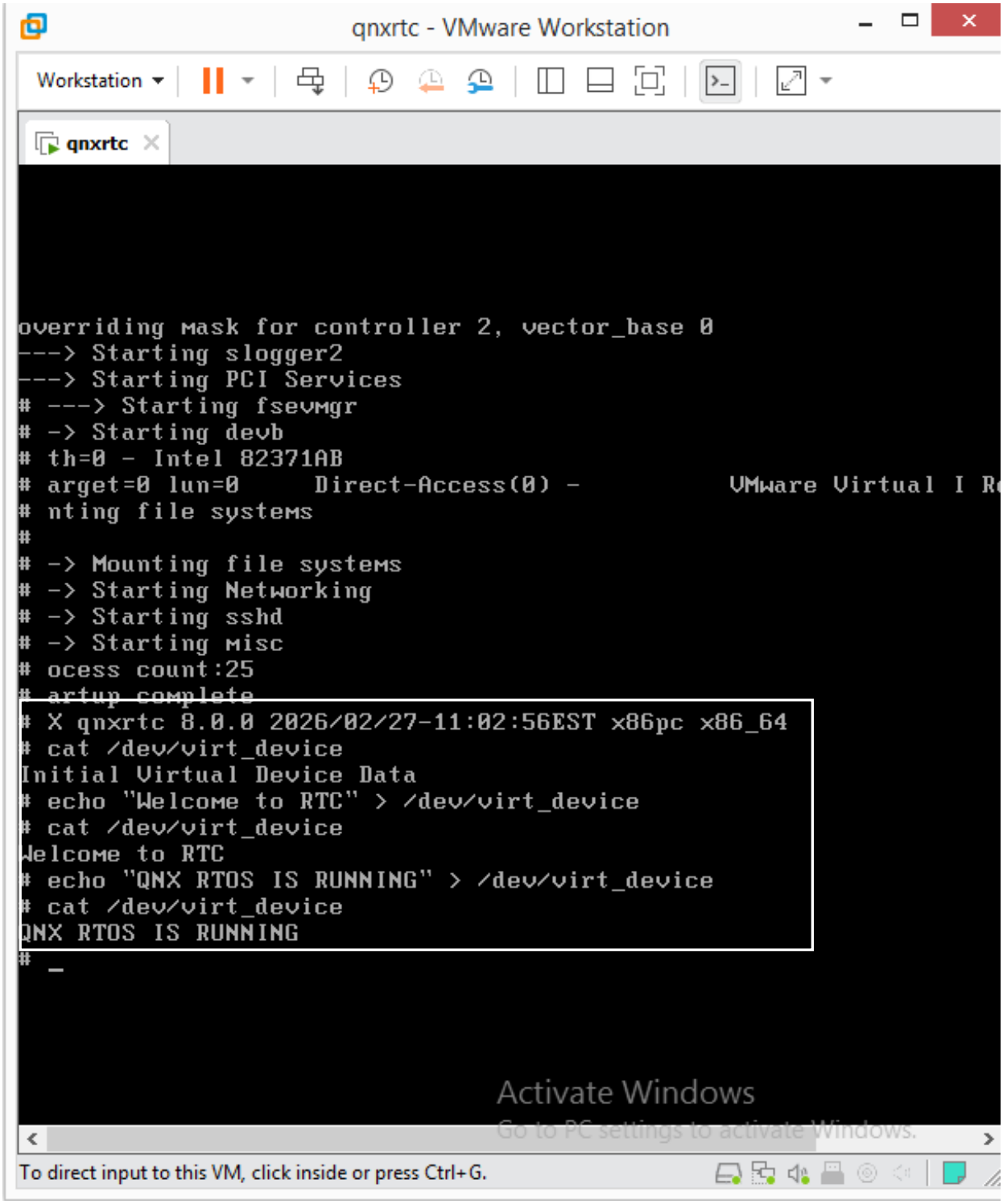
Step 5: Callback Initialization and Override - Executes iofunc_func_init() to populate the driver's handler vectors. It replaces the default read/write points with custom implementations: io_funcs.read = io_read and io_funcs.write = io_write.

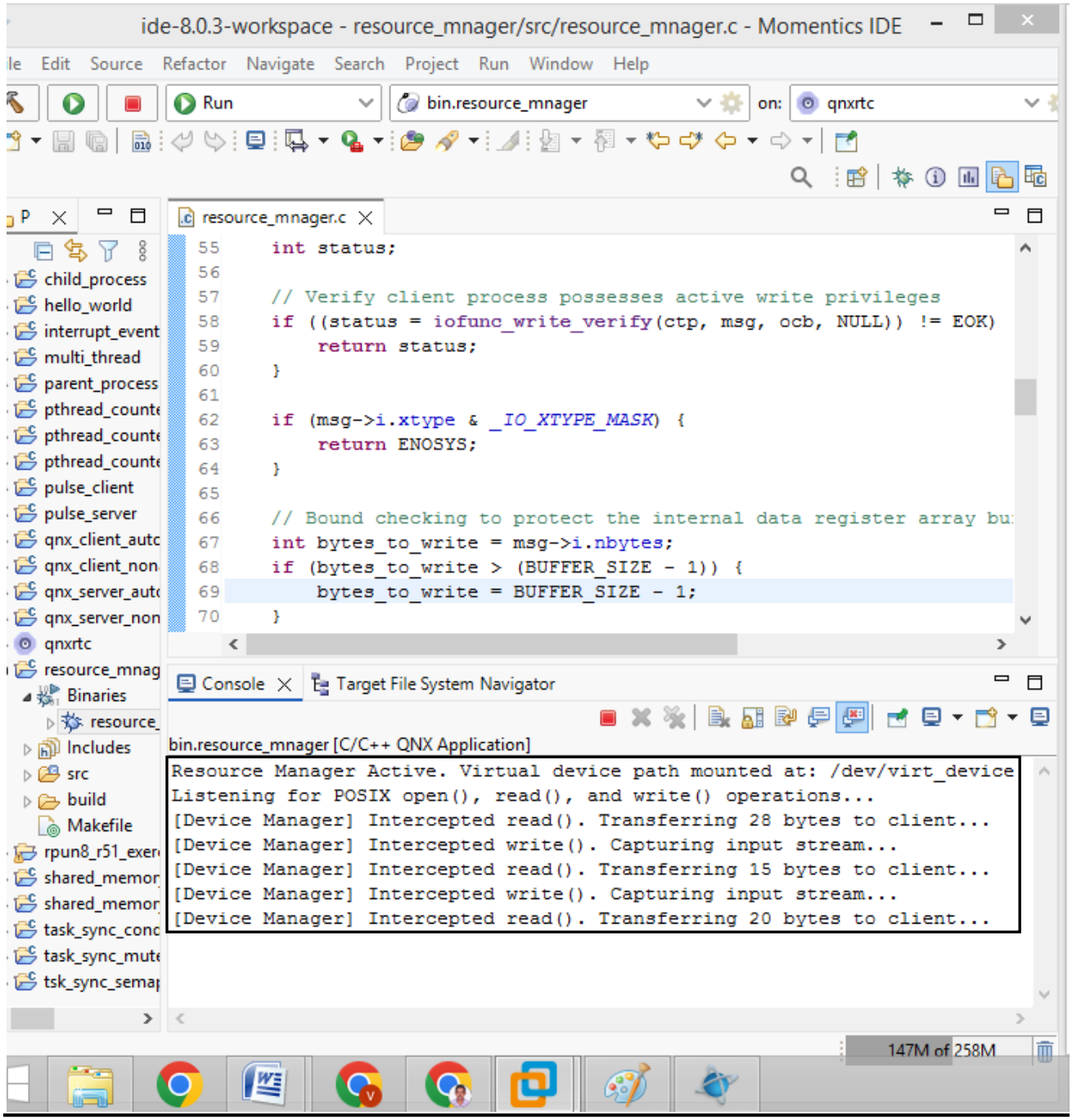
Step 6: Device Registration - Registers the virtual entry string into the target node table via resmgr_attach() at path location /dev/virt_device.

Step 7: Drop into Passive Listener Loop - Enters a while(1) processing loop and blocks on resmgr_block(). The kernel puts the manager thread to sleep until an external client issues an I/O request.

```
while (1)
    {
        if ((ctp = resmgr_block(ctp)) == NULL)
        {
            perror("resmgr_block failed");
            exit(EXIT_FAILURE);
        }
        resmgr_handler(ctp); // Decode message frames and call io_read/io_write
    }

    return 0;
}
```

Output (Terminal 2 as client):




ide-8.0.3-workspace - resource_mnager/src/resource_mnager.c - Momentics IDE
Edit Source Refactor Navigate Search Project Run Window Help
Run
bin.resource_mnager
on:
qnxrtc
resource_mnager.c
int status;
// Verify client process possesses active write privileges
if ((status = iofunc_write_verify(ctp, msg, ocb, NULL)) != EOK)
return status;
if (msg->i.xtype & _IO_XTYPE_MASK) {
return ENOSYS;
// Bound checking to protect the internal data register array bu
int bytes_to_write = msg->i.nbytes;
if (bytes_to_write > (BUFFER_SIZE - 1)) {
bytes_to_write = BUFFER_SIZE - 1;
child_process
hello_world
interrupt_event
multi_thread
parent_process
qnxrtc
resource_mnag
Binaries
Includes
src
build
Makefile
Console
Target File System Navigator
bin.resource_mnager [C/C++ QNX Application]
Resource Manager Active. Virtual device path mounted at: /dev/virt_device
Listening for POSIX open(), read(), and write() operations...
[Device Manager] Intercepted read(). Transferring 28 bytes to client...
[Device Manager] Intercepted write(). Capturing input stream...
[Device Manager] Intercepted read(). Transferring 15 bytes to client...
[Device Manager] Intercepted write(). Capturing input stream...
[Device Manager] Intercepted read(). Transferring 20 bytes to client...
147M of 258M

Procedure:

Step 1: Create a new "QNX Executable" project in Momentics IDE.

Step 2: Declare <sys/iofunc.h> and <sys/dispatch.h> to access the QNX resource manager dispatching and POSIX I/O callback libraries.

Step 3: Call ThreadCtl(_NTO_TCTL_IO_PRIV, 0) in main() to grant the application I/O privileges, allowing it to register paths globally under /dev/.

Step 4: Enforce SCHED_RR (Round Robin) with a priority of 10 to ensure predictable execution time-slicing.

Step 5: Allocate and configure dispatch handles using dispatch_create(). Then, initialize the standard connect and I/O callback tables using iofunc_func_init().

Step 6: Bind custom function handlers to the I/O table vectors—specifically mapping io_read and io_write to custom implementation functions.

Step 7: Register a specific device string name (e.g., "/dev/virt_device") using resmgr_attach(). This mounts the device node into the QNX pathname space.

Step 8: Drop the driver thread into an infinite loop running resmgr_block() and resmgr_handler() to process client I/O events continuously.

Step 9: Build the executable, transfer it to the target, log in as root, and execute it. In a separate target terminal terminal window, issue standard shell commands like echo and cat to verify interaction with the virtual device file.

Results:

Thus, the design and implementation of a basic Resource Manager for virtual device path access under Round Robin scheduling was successfully completed. The program successfully mounted a custom virtual driver file under /dev/virt_device inside the QNX root namespace directory. Testing via standard console utilities verified that user-space system calls (read and write) were correctly intercepted, decoded, and serviced across separate process layers without kernel-level copying or data corruption, proving the modular efficiency of the QNX user-space driver architecture.

Build Script:

```
[virtual=x86_64,bios] .bootstrap = {
    startup-x86
    PATH=/proc/boot
    LD_LIBRARY_PATH=/proc/boot:/lib
    procnto-smp-instr
}
[autoso=list]

[+script] .script = {
    procmgr_symlink /proc/boot/ldqnx-64.so.2 /usr/lib/ldqnx-64.so.2
    display_msg WELCOME TO RTC _ QNX RTCe
    display_msg EXPERIMENT OUTPUT

    devc-con -n4 &
    # start some shells
    reopen /dev/con4
    [+session] ksh &
    reopen /dev/con3
    [+session] ksh &
    reopen /dev/con2
    [+session] ksh &
    reopen /dev/con1
    [+session] ksh &
    hello Welcome to Advanced Embedded Systems Laboratory
}

libc.so
libgcc_s.so.1
ldqnx-64.so.2
libsecpol.so
# drivers
devc-con
# a few handy executables
pidin
ksh
toybox

# toybox needs several shared objects
libregex.so
libfsnotify.so
/lib/libm.so.3=libm.so.3
libm.so
libz.so

# setup common Unix utilites as links to toybox
[type=link] ls=toybox
[type=link] cat=toybox
[type=link] rm=toybox

# include hello executable locally
./hello
```

Expt. No.: 13 BUILDING AND DEPLOYING QNX BOOT/OS IMAGES

Date:

Aim:

To configure, compile, and build an advanced QNX Build File (.build) containing multiple virtual terminal execution sessions (devc-con), embed environment configurations, resolve dynamic linker libraries (ldqnx-64.so.2), incorporate a multi-utility binary (toybox), generate a target-specific boot image (.ifs), and verify the system layout context..

Hardware Requirement:

- Personal Desktop Computer – i7 10th Gen Intel or later
- Raspberry Pi 5 Board with Power Adaptor and Ethernet Cable (Optional)
- Standard AT Keyboard / Emulated Virtual Keyboard

Operating Systems:

- Windows/Linux on Host
- QNX Neutrino RTOS on Target

Software Requirements:

- QNX Momentics IDE 8.0.
- VMware Workstation 17 Pro.
- Target Operating System: QNX Neutrino RTOS (compatible with SDP 8.0).

Algorithm:

Request Interception: When a client app uses echo or write() on the device file, an incoming write message unblocks the manager thread.

Step 1: Start - Open your existing myqnx_rtos_image system project workspace configuration under Momentics IDE.

Step 2: Modify Build Profiles: Update the src/build text data sequence matching the structured multipath solution template mapped out above.

Step 3: Declare Core System Targets: Retain the virtual x86 parameters using procnto-smp-instr to configure real-time symmetric multiprocessing virtualization parameters.

Step 4: Link System Dynamic Libraries: Direct the procmgr_symlink loader process to safely point your shared library objects (/usr/lib/ldqnx-64.so.2) into runtime boot storage arrays (/proc/boot/ldqnx-64.so.2).

Step 5: Configure Multi-Console Workspace Environments: Instantiate the virtual terminal loop mechanism (devc-con -n4 &) and branch four independent processes using sequential pipeline commands (reopen /dev/conX $\rightarrow$ [+session] ksh &).

Step 6: Map Embedded Utilities: Map token names for the compressed core framework module toybox, and explicitly register structural filesystem aliases using the configuration marker [type=link].

Step 7: Verify Binary Output Location Paths: Update the physical source parameter path for the hello execution component targeting your machine's workspace variables.

Step 8: Compile and Output: Run the Clean Project sequence followed by the Build Project task workflow inside the IDE menu workspace to output the unified, deployable .raw/.ifs system file block.

Procedure:

Step 1: Provision an Expanded Virtual Machine

1. In the Target Navigator view of Momentics, right-click an empty area.
2. Select New QNX Target $\rightarrow$ Create a QNX Virtual Machine Instead.
3. In the Extra Options field, type: --boot-size=100 and finish the wizard.

Step 2: Configure the Target Buildfile

1. Open your images project's buildfile in the text editor.
2. Inside the [+script] block, add your display message and automatic application trigger:
   display_msg WELCOME TO RTC _ QNX RTCe
   display_msg EXPERIMENT OUTPUT
3. Inside the file tree manifest section, add the mapping to pull the binary from the workspace ./hello
4. Save the file (Ctrl + S), right-click the project root folder, and select Build Project to generate the .raw image.

Step 3: Deploy the Compiled Image

1. Open the Target Filesystem Navigator view.
2. Locate and expand the target destination directory: /boot/.boot.
3. Drag and drop your newly generated .raw image file from your host workspace straight into that folder.

Step 4: Synchronize & Boot

1. In the target machine terminal console prompt, execute the secure shutdown utility: shutdown.
2. Once the virtual machine restarts, select your new custom image from the boot loader menu list to verify your startup messages and application output.

Results:

The compilation and construction of an advanced multi-console QNX RTOS Boot Image file layout system was successfully achieved. The implementation verified that multiple virtual terminal instances (/dev/con1-4) can run concurrently under isolated shell sessions managed by the devc-con subsystem. Additionally, the experiment confirmed that tool compilation sizes can be optimized via structural alias paths linked directly into a unified toybox compilation framework.